

DUANCNTT_521h0129_521h03 79_final.pdf

bởi Ngô Anh Tuấn

Ngày Nộp: 05-thg 1-2026 12:22CH (UTC+0700)

ID Bài Nộp: 2852785803

Tên Tập tin: DUANCNTT_521h0129_521h0379_final.pdf (1.6M)

Đếm từ: 13647

Đếm ký tự: 84715

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



**DEVELOPING A MULTIMODAL
QUESTION-ANSWERING SYSTEM
BASED ON RAG IN THE FIELDS OF
TRANSPORTATION, CULTURE AND
TOURISM
INFORMATION TECHNOLOGY PROJECT
SOFTWARE ENGINEERING**

Instructor: TS. TRAN THANH PHUOC

Student: TRAN NGOC PHAT - 521H0129

NGO ANH TUAN - 521H0379

Year: 2025

HO CHI MINH CITY, 2025

**VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



**DEVELOPING A MULTIMODAL
QUESTION-ANSWERING SYSTEM BASED
ON RAG IN THE FIELDS OF
TRANSPORTATION, CULTURE AND
TOURISM
SOFTWARE ENGINEERING**

TRAN NGOC PHAT - 521H0129

NGO ANH TUAN - 521H0379

Advised by:

TS Tran Thanh Phuoc

HO CHI MINH CITY, 2025

ACKNOWLEDGEMENT

I would like to express my sincere thanks to the teachers and lecturers of Ton Duc Thang University and especially **Mr. Tran Thanh Phuoc** - lecturer of the Introduction to Information technology project for kindly helping and supporting as well as answering our questions throughout the process of making this essay, so that we can get the best results.

I would like to express my gratitude to **Mr. Tran Thanh Phuoc**, who taught and worked tirelessly, providing me with the necessary tools and skills to complete this report. He played an important role in improving understanding of mathematical logic. The second thing I would like to give to the teachers of the Department of Information Technology of Ton Duc Thang University for giving me the opportunity to do this report, because it is not only an information technology project but also a very important experience for me in the year ahead.

THE PROJECT WAS COMPLETED

AT TON DUC THANG UNIVERSITY

I would like to assure you that this is my project, guided by **Mr. Tran Thanh Phuoc**.

The research contents and results in this topic are original and have not been published in any form before. The data in the tables for analysis, comments and evaluations, collected by the author himself from various sources are clearly cited in the references section.

In addition, the project also uses some comments, reviews as well as figures of other authors and other organizations with quotes and annotations of origin.

If any fraud is detected, we would like to take full responsibility for the content of our project. Ton Duc Thang University is not involved any copyright or issues or violations caused by me during the process (if any).

Ho Chi Minh City, 13 May 2025

TUAN PHAT

NGO ANH TUAN TRANNGOC PHAT

ABSTRACT

This thesis addresses the challenge of “knowledge hallucination” in Large Language Models (LLMs) when retrieving highly localized, culturally specific data. We propose and implement a Hybrid System architecture integrating the Neo4j Graph Database with Retrieval-Augmented Generation (RAG) techniques, specifically applied to two domains: optimizing public bus transit routes in Ho Chi Minh City and providing in-depth Vietnamese cultural and tourism information.

Methodologically, the system employs a “Logical Partitioning” strategy to handle two distinct data streams. For the transport subsystem, the project implements a Deterministic Path-finding algorithms that utilize the Haversine formula to compute actual edge weights, ensuring absolute accuracy for complex multi-hop itineraries. For the culture and tourism subsystem, to handle unstructured data rich in historical semantics, the system deploys a Hybrid Search mechanism that combines exact keyword matching with semantic vector search (utilizing the domain bkai-foundation-models embedding). This approach enables the system to deeply comprehend user intent even with abstract or ambiguous queries.

The system uses a FastAPI Backend and Streamlit Frontend, integrating the Google Gemini 2.0 Flash model to synthesize natural language responses from multimodal queries (text and image). Experimental results demonstrate the the system not only provides optimal travel itineraries but also functions as a virtual guide, delivering verified cultural insights with vivid illustrations, effectively mitigating the lack of knowledge depth typical found in conventional chat-bots.

CONTENT TABLE

ACKNOWLEDGEMENT	i
THE PROJECT WAS COMPLETED	ii
AT TON DUC THANG UNIVERSITY	ii
ABSTRACT	iii
CONTENT TABLE	iv
LIST OF IMAGE	vii
LIST OF TABLE	vii
LIST OF ABBREVIATIONS	ix
CHAPTER 1: INTRODUCTION	1
1.1 Implementation Context (Focusing on Transport, Culture, and Tourism)	1
1.2 Project Objectives (Multimodal Optimization)	2
1.3 Scope of the Project:	3
CHAPTER 2 BACKGROUND KNOWLEDGE	5
2.1 Context	5
2.2 Representative Internation Works	5
2.3 Novelty and User Appeal	8
CHAPTER 3 THEORETICAL OVERVIEW	10
3.1 Large Language Models (LLMs)	10
3.2 Overview of Knowledge Graphs:	16
3.3 Retrieval-Augmented Generation (RAG):	17

3.4 Retrieval	21
3.5 Generation:	24
3.6 Optimzation and Challenges in RAG	29
CHAPTER 4 PROPOSED METHOD	32
4.1 Motivation for the Contribution:	32
4.2 Building a multimodal Q&A system based on RAG in the field of Transportation - Culture - Tourism	34
CHAPTER 5 SYSTEM CONSTRUCTION AND ALGORITHM	49
5.1 Knowledge Graph Construction and Implementation:	49
5.2 Routing Algorithms and Query Processing Workflow	50
5.3 System Implementation Technologies and Architecture	51
CHAPTER 6 EXPERIMENTAL RESULTS AND EVALUATION	56
6.1 Analysis of Testing Scenarios:	56
6.2 Phases 1: Baseline Assessment and Vulnerability Detection (Dataset V1)	57
6.3 Optimization Measures:	58
6.4 Experimental Conclusion:	59
CHAPTER 7 APPLICATION ILLUSTRATION	60
7.1 Scenario 1: Public Transport Routing (Bus Routing)	60
7.2 Scenario 2: Tourism Q&A (Tourism)	61
7.3 Scenario 3: Multimodal Processing (Multimodal Input)	63
7.4 Scenario 4: Complex Planning (Reasoning):	63
CHAPTER 8 CONCLUSION AND FUTURE DEVELOPMENT	65

8.1 Generation Conclusion:	65
8.2 Future Work	65
REFERENCES	67

LIST OF IMAGE

Image 1 : Flowchart of the PDF-to-Text conversion process	37
Image 2 Workflow of the LLM-based Text-to-JSON extraction process.	40
Image 3 Overview of the Data Preprocessing Workflow.	44
Image 4 A sample of the resulting JSON (hcm_bus_final.json).....	45
Image 5 Preprocessing Workflow for Bus Route Data	48
Image 6 Integrated Transport and Tourism Graph Schema Database.	49
Image 7 The logic diagram for handling state and events in the Streamlit frontend.	54
Image 8 UI Mutimodal System	55
Image 9 The illustration shows the system for selecting a specific destination for the user.	60
Image 10 The illustration shows the system finding the route from Ben Thanh Market to Suoi Tien using bus number 19.	61
Image 11 Illustration of the tourism Q&A feature. The response is clearly structured with headings and bullet points.(part 1).....	62
Image 12 Illustration of the tourism Q&A feature. The response is clearly structured with headings and bullet points. (part 2)	62
Image 13 Illustration of image recognition and context-aware recommendation. ...	63
Image 14 Illustrating the functionality of complex travel planning (Part 1: Identifying requirements and suggesting morning itineraries).	64
Image 15 Illustrating the features of complex travel planning (Part 2: Detailed afternoon itinerary and next-mile travel instructions).	64

LIST OF TABLE

Table 1 Phase 1 (Baseline) Test Results	57
Table 2 Phase 2 Experimental Results	58

LIST OF ABBREVIATIONS

LLM	Large Language Model
QA	Question and Answer
POis	Points of Interest
RAG	Retrieval-Augmented Generation
KG	Knowledge Graph
NLP	Natural Language Processing

CHAPTER 1: INTRODUCTION

1.1 Implementation Context (Focusing on Transport, Culture, and Tourism)

In the context of international integration and the rapid growth of smart tourism, demand for integrated, accurate access to information on Transport, Culture and Tourism is increasing. Modern users require a tool that can answer complex, cross-domain questions such as: “How can I travel from Airport A to Heritage Site B by public transport, and what cultural etiquette should I be aware of when visiting?”

This domain presents many challenges that make traditional QA systems struggle:

- **Multimodality:** Data is not only text (location descriptions, traffic regulations) but also images (maps, traffic signs, destination photos). Traditional text-only systems cannot answer visual-recognition-based questions.
- **Cross-domain and Complex Relations:** Transport information (signs, routes) is tightly coupled with Culture (festivals, customs, etiquette) and Tourism (POIs, services). Queries often require logical linking between entities (e.g., “Location A can be reached via Transport Route B during Festival C”).
- **Limitations of Generic Chat-bots (Hallucination):** General-purpose LLMs often lack deep, precise knowledge of local traffic laws, road signs, or specific cultural practices in Vietnam. This leads to hallucinations, especially for highly dynamic or localized information.

→ **Proposed Solution: “Multimodal RAG combined with a Knowledge Graph”**

To address the complexity and diversity of this domain, we propose an advanced architecture: **Multimodal RAG** integrated with a **Knowledge Graph (KG)**.

- **Multimodal RAG:** Uses multimodal embedding models to encode and store both text and images in a joint vector space. This enables retrieval not only based on textual semantics but also on visual content.
- **Knowledge Graph (KG):** Core component to manage logical relations among Transport entities (road signs, routes), Culture (events, rules, etiquette), and Tourism (POIs). KG supports multi-step reasoning and structured querying (e.g., retrieving all historically significant attractions in Ho Chi Minh City).

Practical Significance

The system is expected to:

- Increase accuracy and coverage for cross-domain questions by aggregating information from text, images, and structured relations.
- Assist tourists and citizens in trip planning, learning about culture, and discovering attractions effectively.
- Provide a structured knowledge infrastructure for the Transport – Culture – Tourism domain in Vietnam.

1.2 Project Objectives (Multimodal Optimization)

1.2.1 Main object:

- Develop a Vietnamese Multimodal QA Chatbot for Transport – Culture – Tourism based on a Multimodal RAG architecture integrated with a Knowledge Graph.
- Build a Multimodal Data Repository and Knowledge Graph for this domain.

1.2.2 Specific objectives:

- Process and normalize multimodal data (Text & Image) from Transport, Culture, and Tourism sources.

- Perform Entity & Relation Extraction to construct a Transport – Culture – Tourism Knowledge Graph.
- Develop a QA model based on Multimodal RAG combined with KG (e.g., using Fusion Retrieval to combine vector search and graph querying).
- Implement a demo UI that accepts multimodal input (text and images) and outputs text, accompanied by related images/maps.

1.3 Scope of the Project:

Spatial scope: District 1 and 3 Ho Chi Minh City.

Rational: High POI density (historical sites, museums, attractions; dense bus route – bus stop network); relatively available open data; high query demand from foreign visitors to HCMC or people who want to explore itineraries, culture, relics, museums via MM-RAG integrated with a Knowledge Graph, answer generation with citations and within a static, pre-defined scope.

Content scope:

- Transport (static): Bus routes and stops within/through Districts 1 and 3; reference itineraries; descriptive information on operating hours and (non-real-time) frequencies.
- Culture – History: Independence Palace, Notre-Dame Cathedral Basilica of Saigon, Central Post Office, Opera House, City Hall, Bến Thành Market, Vietnam History Museum, War Remnants Museum, Ho Chi Minh City Museum of Fine Arts.
- Tourism – Food – Nearby Experiences: Recommendations for nearby POIs within a radius/time budget; neutral descriptions; trip planning. Example: “Plan a 3 days 2 nights trip in Ho Chi Minh City”, “Instagrammable spots loved by young people”.

- Out of scope: Real-time data (ETA, traffic congestion, dynamic prices), ticket booking/promotions; advertising-oriented comparisons/rankings.

Languages: Vietnamese.

CHAPTER 2 BACKGROUND KNOWLEDGE

2.1 Context

Given the complexity of the Transport – Culture – Tourism domain, QA systems must go beyond plain-text retrieval. The project applies two advanced trends:

- Multimodal RAG: Extends RAG from purely textual processing to Text + Image, enabling the model to understand visual context (e.g., traffic signs, landmark photos) when answering.
- KG-RAG: Integrates a Knowledge Graph to support queries requiring logical reasoning, route finding, and cross-domain relations—areas where pure vector search alone is limited.

2.2 Representative Internation Works

- Gao et al., 2023 – “Retrieval-Augmented Generation for LLMs: A Survey” A comprehensive survey of RAG evolution, from basic retriever-generator architectures to advanced variants like multi-hop, fact-augmented, and KG-RAG. The authors decompose RAG into three pillars (retrieval, augmentation, generation), discuss how to combine unstructured data with structured knowledge (KGs), and compare RAG vs. fine-tuning in terms of efficiency, scalability, and reliability. [1]
- RA-CM3 (Retrieval-Augmented CM3, ICML 2023 – Yasunaga et al.) Proposes a multimodal RAG model combining images and text. CLIP (bi-encoder) is used to retrieve related documents (both images and text), while CM3—a Transformer that can generate both images and text—acts as the generator. RA-CM3 can generate with mixed text and images, outperforming earlier multimodal models (e.g., DALL·E, original CM3) on image generation and captioning tasks (improvements of ~12 FID and ~17 CIDEr on MS-COCO) while reducing training cost. [2]

- Hogan et al., 2021 – Survey on Knowledge Graphs Provides a comprehensive overview of Knowledge Graphs: data models (RDF, property graphs), query languages (Cypher, Gremlin), deductive & inductive reasoning techniques, and Knowledge Graph Completion methods (classical vs. representation-learning-based). Also outlines challenges and future directions. [3]
- TravelRAG (IJGI, 2024) Addresses POI retrieval based on a multi-layer KG, merging data from websites and social platforms. Travel-KG serves as the core retrieval source, hierarchically organizing destinations and evaluations. [4]
- He et al., 2024 Targets large-scale textual graphs, proposing retrieval of relevant subgraphs via PCST optimization, then combining GNN + LLM for reasoning and answer generation. Demonstrates advantages on multi-hop questions, scaling to large graphs, and reducing hallucinations. [5]
- Zhu et al., 2025 – KG2RAG proposes a model combining semantic retrieval with graph-guided chunk expansion (adding extra chunks related to the question via KG relations), which improves the diversity and coherence of answer content. [6]
- Hur et al., 2021 – Survey on automatic KG construction Classifies techniques by data source (text, tables, web) and method (information extraction, entity linking, knowledge fusion). Highlights challenges such as handling noisy data, knowledge updates, and KG quality evaluation. [7]
- Mukherjee et al., 2025 Introduces a KG-RAG system for enterprise document chatbots with high-quality KG construction for RAG, reducing irrelevant responses by 50% and increasing fully-satisfactory responses by 88%. [8]
- “Knowledge Graph-Guided Retrieval Augmented Generation (KG2RAG)” (2025) Uses KG to expand and organize retrieved content, improving coherence and diversity on HotpotQA. [9]

- “KG Retrieval-Augmented Generation for LLM-based Recommendation”
(2025)Applies KG-RAG to enhance recommender systems, reducing hallucinations and improving knowledge updates. [10]
- Survey on Multimodal Knowledge Graphs (MMKG, 2023–2024)Overviews
MMKGs that combine images + text as KG attributes. In our context, MMKG can attach maps/images directly as attributes in the KG; RAG then retrieves subgraphs and documents simultaneously to generate multimodal evidence-based answers. [11]
- “Document GraphRAG: Knowledge Graph Enhanced RAG for Document QA in the Manufacturing Domain” (2024)Surveys RAG enhancements by integrating document structure and KG to handle long, multi-source information in industrial settings. [12]
- Fan et al., 2024 – CuPe-KG (Cultural Perspective-based KG for Tourism Resources)
Organizes heritage and cultural knowledge by cultural perspective, using PLMs for entity & relation extraction and perspective labeling. The resulting KG provides rich context for QA about events, artifacts, and sites. [13]
- Wang et al., 2017 – Knowledge Graph Embedding SurveyReviews translation-based models (TransE, TransH, TransR) and semantic matching models, analyzing pros/cons and applications such as link prediction and entity classification. [14]
- Sammaritin et al., 2024 – KG-RAG: Bridging Knowledge and CreativityUses KG-guided Chain of Explorations to structure retrieval paths in KG, then feeds them to LLMs for more accurate responses. [15]
- Swacha Gracle, 2025 – Survey of RAG-based chatbots in educationLists systems like ChatPapers, ChatDoctor, ChatLaw, PaperQA combining LLMs with RAG in specialized domains. [16]

- Riedler & Langer, 2024 – Beyond Text: Optimizing Multimodal RAGIndustrial study comparing MM-RAG configurations; concludes that “caption-first indexing” (captioning/summarizing images before indexing) is often more robust and effective than using raw multimodal embeddings for retrieval. [17]
- Karpukhin et al., EMNLP 2020 – Dense Passage Retrieval (DPR)A foundational dense retrieval method for open-domain QA: dual-encoder (query/doc) with ANN index, improving top-20 accuracy by 9–19% over BM25 on multiple datasets. In our project, DPR serves as the dense-text retrieval backbone, combined with reranking/graph-based constraints to balance recall and faithfulness. [18]
- Xu et al., 2024 – KG-RetrievalProposes a RAG framework with hierarchical indexing based on KG and documents, improving multi-document retrieval performance. [19]

Representative Domestic Works

- Ho Chi Minh City University of Technology (HCMUT), 2024 Built a KG from multiple data sources in the education domain and integrated it into a QA system combining LLMs and RAG. This is one of the few practical Vietnamese works applying RAG + KG. [20]

2.3 Novelty and User Appeal

Our project combines Multimodal RAG (Text + Image) with KG in the specific context of HCMC (Districts 1–3, Vietnamese), emphasizing image-to-text summarization and multi-vector retrieval constrained by KG with mandatory clause-level citation.

Compared with the works above, the key differences are:

- High degree of localization (HCMC cultural–tourism–transport context).
- Strict citation requirements and evaluation framework separating text/image & faithfulness – suitable for local cultural–tourism applications.

Natural, urban heritage-aware experience (Text + Image):

Users can ask via text or send photos of architectural heritage. Supporting on-site interactions allows the system to “hit the real-world context” at points of interest, unlike pure text QA.

Strong localization:

Data is sourced from official, detailed HCMC resources, especially key districts (1 & 3). Default language is Vietnamese, making responses close to real-world usage compared with international systems that do not understand local context.

Serving multiple user groups:

Domestic and international tourists, students, and teachers needing cultural reference materials, etc.

CHAPTER 3 THEORETICAL OVERVIEW

3.1 Large Language Models (LLMs)

A Large Language Model (LLM) is a type of AI model developed to process and generate human-like natural language. LLMs are based on deep learning, especially Transformer architectures, to capture language structure and semantics. Trained on massive corpora (books, articles, websites, etc.), LLMs can model complex patterns, context, semantics, and even cultural nuances across languages.

Technically, LLMs operate on next-token prediction, leveraging context. Thanks to the self-attention mechanism in Transformers, they can identify important parts of the text regardless of sequence length, learning relationships between words/phrases to generate coherent, context-appropriate content.

3.1.1 Core Components:

- **Deep Neural Network (DNN):**

Multi-layer neural network backbone, enabling large-scale pattern recognition in syntax and semantics.

- **Training Data:**

Massive datasets from books, articles, websites, research papers, etc. Billions of sentences provide linguistic and cultural patterns, making LLMs versatile.

- **Optimization Algorithms:**

Gradient descent-based methods (e.g., Adam, SGD) update millions to billions of parameters to minimize loss and improve prediction accuracy.

- **Attention Mechanism:**

Crucial for modeling relationships between tokens in long sequences. Self-attention lets Transformers focus on relevant positions, improving accuracy and efficiency.

3.1.2 Operating Principle:

The working principle of a Large Language Model revolves around the ability to predicting the next word in a sentence based on previous context. This is done through the following key components and techniques:

- **Next-Word Prediction:**

LLMs learn to predict the next word in a text sequence by analyzing the context of preceding tokens. This process enables the model to generate coherent sentences that respect grammar and semantics.

For example, in the sentence “Hôm nay trời rất...” (“Today the weather is very...”), the model will likely predict “đẹp” (“nice”) or “nóng” (“hot”) based on patterns learned from the training data.

- **Transformer Neural Network:**

The Transformer is the neural network architecture that underpins most modern LLMs. Its strength lies in its ability to process long, complex sequences, allowing the model to capture relationships between words across an entire paragraph rather than only within a local window.

- **Self-Attention Mechanism:**

The self-attention mechanism allows the model to focus on the most relevant parts of the sequence when processing language. Instead of relying solely on token positions, self-attention evaluates relationships between all tokens in a sentence, regardless of their distance. This is especially useful for handling long, syntactically or semantically complex sentences.

- **Learning from Large-Scale Data:**

LLMs are trained on massive corpora, often containing billions of tokens from diverse sources. During training, the model continually adjusts its parameters using optimization algorithms to improve prediction accuracy and language modeling capabilities.

3.1.3 Advantages and Limitations of LLMs:

Advantages:

- **Natural language understanding and generation:** LLMs can understand and generate text with quality approaching human-level, supporting tasks such as question answering, content generation, and academic writing.
- **Large-scale data processing:** These models can analyze and learn from billions of words, enabling them to capture complex linguistic patterns and perform a wide range of NLP tasks.
- **Creative content generation:** LLMs can generate novel, contextually appropriate text for tasks such as article writing, script generation, and storytelling.
- **Automation of many tasks:** LLMs can help automate work such as report writing, information summarization, and customer support, thereby saving time and increasing productivity.

Limitations:

- **High computational cost:** Training and deploying LLMs require powerful and expensive hardware (e.g., GPU/TPU clusters), especially in production environments.
- **Possibility of errors and inaccuracies:** Although LLMs can generate fluent natural language, they may still produce incorrect, misleading, or logically inconsistent information.

- **Lack of deep semantic understanding:** LLMs do not truly “understand” meaning in the human sense. They operate based on statistical patterns learned from training data rather than genuine reasoning.
- **Ethical and bias-related issues:** LLMs can inherit biases present in training data, leading to unfair, discriminatory, or skewed content generation if not carefully controlled.

3.1.4 Training Procedure for Popular Large Language Models

Step 1: Collecting Training Data

Data is the fundamental component for training LLMs. Sources typically include books, articles, websites, social media, and domain-specific documents. Raw data is cleaned and normalized using Natural Language Processing (NLP) techniques to ensure quality and correctness before being fed into the model.

Step 2: Designing the Model Architecture

Most modern LLMs adopt the Transformer architecture with an attention mechanism, especially self-attention, to model contextual relationships between tokens. Hyperparameters such as the number of layers, embedding dimension, and number of attention heads are tuned to optimize model performance.

Step 3: Training with Deep Learning

The training process uses gradient-based optimization (e.g., Gradient Descent) to adjust the neural network weights. Optimizers such as Adam or Stochastic Gradient Descent (SGD) are used to minimize the loss function, helping the model accurately predict the next word or sequence.

Step 4: Leveraging High-Performance Computing Systems

LLMs require powerful hardware such as GPUs or TPUs to process billions of parameters. To accelerate training, models are often trained on distributed clusters, where data and computation are sharded across multiple machines.

Step 5: Fine-Tuning

After the initial pretraining phase on large general-purpose corpora, the model is fine-tuned on task-specific or domain-specific datasets to adapt it to concrete applications such as text classification, summarization, or domain-specific text generation.

Step 6: Evaluation and Testing

The performance of an LLM is evaluated by comparing its outputs against a held-out test set. Metrics such as BLEU, ROUGE, and perplexity are commonly used to assess the quality and fluency of generated language.

Step 7: Deployment and Continuous Updates

Once trained and validated, the model is deployed to applications such as chatbots, virtual assistants, or search systems. To maintain performance and relevance, LLMs are periodically updated and retrained with new data.

3.1.5 Real-World Applications of LLMs

In Marketing and Advertising

- **Content generation:** LLMs support the creation of high-quality content and provide capabilities such as editing, grammar correction, style refinement, and accurate summarization.
- **Personalized marketing:** LLMs enable content personalization across channels such as email, social media, and digital advertising.
- **Intelligent chat-bots:** LLM-powered chat-bots provide 24/7 customer support, handle requests efficiently, and log interaction data for further analysis.

- **Sentiment analysis:** LLMs can analyze customer sentiment on social media and other platforms, detecting emotions and subtle nuances in user feedback.

In Retail and E-commerce

- **Personalized recommendation:** Large language models can leverage data such as purchase histories and browsing behavior to detect patterns and provide optimized, personalized product or service recommendations. LLMs can also handle complex queries and return more accurate results, enhancing the online shopping experience.

In Education

- **Automated grading and feedback:** LLM-integrated tools help reduce teachers' workloads by providing instant feedback to students. This not only improves the learning experience but also encourages self-study and personal responsibility.
- **Real-time performance tracking:** By analyzing learning data in real-time, LLMs can provide teachers with detailed insights into students' difficulties. This enables timely adjustments in teaching strategies, remediation of knowledge gaps, and overall performance improvement.

In Healthcare

- **Medical consultation and triage support:** Effective communication is crucial in healthcare, where patients require timely advice and support. LLMs serve as the backbone of intelligent chatbots that can understand and answer patient questions in a natural and accurate manner.
- **Enhancing Electronic Health Records (EHR):** As EHRs become more prevalent, the volume of clinical data increases dramatically. LLMs support the analysis of clinical notes and the extraction of valuable information from case reports, helping organize and classify data more effectively. This assists healthcare professionals in identifying anomalies, reducing errors, and making more accurate diagnoses.

3.2 Overview of Knowledge Graphs:

A Knowledge Graph (KG) is a knowledge base represented in the form of a graph structure. This structure is commonly used to store interconnected descriptions of entities such as objects, events, situations, or abstract concepts, while encoding their semantics and the core relationships between these entities.

In other words, a KG models the world as:

- Nodes (entities): things like places, people, events, organizations, concepts
- Edges (relations): semantic links between entities, such as “located in”, “part of”, “happens at”, “belongs to”, etc.

3.2.1 Graph Theory:

Formally, a graph is defined as a pair (V, E) where:

- V (Vertices): the set of vertices or nodes, each representing an entity in the graph.
- E (Edges): the set of edges or links connecting pairs of vertices, representing relationships between entities.

Edges in a graph can be directed or undirected:

- Undirected edge: An undirected edge connects two vertices A and B such that one can move from A to B and from B to A . The relationship is symmetric (e.g., “connected to” in an undirected network).
- Directed edge: A directed edge represents a one-way relationship: you can traverse from vertex A to vertex B , but not necessarily in the reverse direction. This is suitable for asymmetric relations such as “ A is located in B ”, “ A causes B ”, or “ A precedes B ”.

In Knowledge Graphs, directed edges are often used to represent semantic relations in as triples: (head entity, relation, tail entity) (“Notre-Dame Cathedral”, “located_in”, “District 1”).

3.3 Retrieval-Augmented Generation (RAG):

3.3.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an advanced technique in Artificial Intelligence (AI) and Natural Language Processing (NLP) that enhances the capabilities of Large Language Models (LLMs) by integrating an external information retrieval mechanism.

Instead of relying solely on the knowledge encoded during pretraining, RAG allows an LLM to search for and leverage relevant information from an external knowledge base to generate responses that are more accurate, up-to-date, and better grounded in factual data.

Traditional LLMs, although capable of generating impressive text, often suffer from the following problems:

- **Hallucination:** Generating incorrect, fabricated, or non-existent information.
- **Stale knowledge:** The model’s knowledge is limited to its training data and cannot reflect the most recent information.
- **Lack of explainability:** It is difficult to trace or justify the source of generated information.

RAG was introduced to address these limitations by providing the LLM with a mechanism to “look up” external information before generating an answer. This significantly improves the accuracy, reliability, and explain ability of model outputs.

3.3.2 Why is RAG important ?

RAG brings several key advantages, especially in applications that require high accuracy and up-to-date information:

- **Improved accuracy and reliability:** By retrieving information from trustworthy sources, RAG reduces hallucinations and ensures that responses are grounded in real data.
- **Continuously updated knowledge:** The model can access the latest information without requiring a full retraining of the LLM, keeping the system aligned with newly added or frequently changing data.
- **Explainability:** RAG enables for users or downstream systems to inspect which documents or passages were used to generate an answer, thereby improving transparency and interpretability.
- **Reduced retraining cost:** There is no need to constantly retrain the LLM on new data; instead, updating or expanding the knowledge base is often sufficient, saving both computation and time.
- **Handling specialized information:** RAG enables LLMs to answer questions in specialized domains or over private, organization-specific data that the model was never explicitly trained on.

3.3.3 Core Components of a RAG system:

A basic RAG system typically consists of two main components working together: Retrieval Module and Generation Module.

3.3.3.1 Retrieval Module:

The Retrieval Module is responsible for searching and retrieving relevant text segments or documents from a large knowledge base in response to the user's query. This process usually includes the following steps:

1. **Query encoding:** The user query is converted into a vector representation (an embedding) using an embedding model.
2. **Similarity search:** The query embedding is used to search for text segments whose embeddings are similar within the knowledge base. This knowledge base is typically stored in a vector database to enable fast, scalable similarity search at scale.
3. **Document retrieval:** The documents or text chunks with the highest similarity scores to the query are retrieved and passed to the Generation Module.

3.3.3.2 Generation Module:

The Generation Module, typically implemented using a Large Language Model (LLM), receives as input: the original user query and the relevant text chunks retrieved by the Retrieval Module. The LLM then uses both sources of information to generate a coherent, accurate, and contextually appropriate response.

This process enables the LLM to:

- **Synthesize information:** Combine its internal parametric knowledge with externally retrieved evidence.
- **Generate natural language:** Produce responses in fluent, human-readable natural language that are easy for users to understand.
- **Provide contextualized answers:** Ensure that the final response is directly relevant to the user's query and grounded in the retrieved documents.

3.3.4 RAG Workflow:

The typical workflow of a RAG system can be summarized as follows:

1. **User query submission:** The user poses a question or issues a request.
2. **Query embedding:** The query is converted into a dense vector representation (query embedding).
3. **Knowledge base search:** The query embedding is used to search for the most relevant text segments in the knowledge base (whose contents have been pre-embedded).
4. **Document retrieval:** Relevant text chunks are retrieved from the knowledge base.
5. **Context augmentation:** The original user query and the retrieved text chunks are combined into a rich input context for the LLM.
6. **Response generation:** The LLM consumes the augmented context and generates the final answer to return to the user.

3.3.5 Conclusion

RAG represents an important step forward in the development of conversational and information-centric AI systems. By combining the generative power of LLMs with efficient information retrieval, RAG unlocks many new applications and substantially improves the performance and reliability of existing AI systems.

Subsequent sections will delve deeper into each component of RAG, implementation techniques, and the key challenges that need to be addressed in real-world deployments.

3.4 Retrieval

3.4.1 Introduction to the Retrieval Module

The Retrieval Module is responsible for searching and extracting relevant information from a large knowledge base in order to provide context for the Large Language Model (LLM). Its primary objective is to ensure that the retrieved documents are high-quality, relevant, and diverse, thereby enabling the LLM to generate accurate and comprehensive responses.

The retrieval process typically involves converting both the user query and the documents in the knowledge base into vector representations (embeddings), then using similarity search techniques to identify the most relevant documents.

The performance of the Retrieval Module directly impacts the overall quality of the outputs produced by the RAG system.

3.4.2 Main Components of the Retrieval Module

3.4.2.1 Knowledge Base

The knowledge base store all documents accessible to the RAG system. It can include:

- **Text databases:** Textual documents such as articles, books, reports, technical documentation, etc.
- **Structured databases:** Data from tables, SQL databases, NoSQL databases.
- **Multimedia data:** Images, videos, audio (after being converted into text or into searchable representations).

To optimize retrieval efficiency, documents in the knowledge base are usually split into smaller chunks and converted into embedding vectors before being stored.

The choice of chunk size is a crucial design decision:

- Chunks that are too large may contain excessive irrelevant information (noise).
- Chunks that are too small may lose important context.

3.4.2.2 Embedding Model:

The embedding model is a critical component that converts text (user queries and document chunks in the knowledge base) into numerical vectors (embeddings) in a high-dimensional space. These vectors are constructed such that semantically similar texts have embeddings that are close to each other in the embedding space.

Common types of embedding models include:

- **Word embeddings:** Word2Vec, GloVe, FastText (represent individual words).
- **Sentence embeddings:** Sentence-BERT, Universal Sentence Encoder (represent entire sentences or paragraphs).
- **Contextual embeddings:** Transformer-based models such as BERT, RoBERTa, or OpenAI Embeddings (generate embeddings conditioned on the surrounding context).

Choosing an appropriate embedding model is extremely important, as it directly affects the system's ability to retrieve truly relevant documents.

3.4.2.3 Retrieval Pipeline:

The retrieval process in a RAG system generally proceeds as follows:

1. Offline document preprocessing and embedding:

- Documents in the knowledge base are collected and cleaned.
- Each document is split into smaller chunks.
- Each chunk is converted into an embedding vector using the embedding model.
- These embedding vectors, together with their corresponding chunk IDs, are stored in a vector database.

2. Online query processing:

- When a user submits a query, it is converted into an embedding vector using the same embedding model as for documents.
- The query embedding is sent to the vector database to search for document embeddings with the highest similarity.
- The vector database returns the IDs of the most relevant document chunks.
- The original document chunks corresponding to these IDs are retrieved from the knowledge base and passed to the Generation Module.

3.4.2.4 Chunking Strategies:

Chunking is one of the most important steps in Retrieval, as it directly affects the quality of retrieved documents. Common chunking strategies include:

- **Fixed-size chunking:** Splitting documents into chunks of fixed size (e.g., 256 or 512 tokens) with a certain overlap between consecutive chunks to preserve context.
- **Semantic chunking:** Splitting documents based on semantic structure, e.g., by paragraphs, headings, or logically complete sections.
- **Recursive chunking:** Splitting documents at multiple hierarchical levels — starting from larger units (e.g., chapters, sections) and then recursively breaking them down into smaller units (e.g., paragraphs, sentences).

3.4.2.5 Challenges in Retrieval:

Key challenges in the Retrieval Module include:

- **Embedding quality:** If the embedding model is not sufficiently good, the system may retrieve irrelevant or low-quality documents.
- **Chunk size selection:** Choosing an appropriate chunk size is non-trivial and directly affects context preservation and noise level.

- **Search over large-scale data:** Ensuring search speed and efficiency when operating over massive knowledge bases.
- **Handling outdated or incorrect information:** Ensuring that the knowledge base is kept up to date and does not contain inaccurate or copyright-violating content.
- **Query ambiguity:** A single query may have multiple interpretations, potentially leading to retrieval of undesired or irrelevant documents.

3.4.2.6 Advanced Retrieval Techniques:

- **Re-ranking:** After an initial set of candidate documents is retrieved, a more sophisticated re-ranking model can reorder them based on more accurate relevance scores, selecting the best subset to feed into the LLM.
- **Hybrid search:** Combining traditional keyword-based search (e.g., BM25) with vector-based semantic search to leverage both precise keyword matching and semantic understanding.
- **Multi-hop retrieval:** For complex questions that require information from multiple sources or multi-step reasoning, the system can perform several rounds of retrieval to progressively gather all necessary evidence.
- **Query expansion / rewriting:** Expanding or rewriting the user query to improve retrieval quality, for example by adding synonyms, related terms, or generating auxiliary sub-queries.

3.5 Generation:

3.5.1 Introduction to the Generation Module:

The Generation Module is responsible for producing the final response to the user. After the Retrieval Module has fetched relevant text chunks from the knowledge base, the Generation Module (typically a Large Language Model – LLM) uses this retrieved

information together with the user's original query to synthesize and generate a coherent, accurate, and natural response.

The objective of the Generation Module is not only to produce fluent text, but also to ensure that the response is grounded in evidence, contextually appropriate, and effectively addresses the user's query.

3.5.2 Main Components of the Generation Module:

3.5.2.1 Large Language Model (LLM):

The LLM is the core of the Generation Module. It is a deep learning model trained on massive text corpora, enabling it to understand, generate, and interact using natural language.

In the context of RAG, the LLM serves as the reasoning and synthesis engine, combining its internal parametric knowledge with external information from the Retrieval Module.

Common types of LLMs used in RAG include:

- **Encoder–Decoder Models:** For example, BART, encodes the input and decodes it into an output sequence.
- **Decoder-only Models:** For example, GPT-4, LLaMA, which are designed to generate text autoregressively based on the given input context.

3.5.2.2 Prompt Engineering:

In RAG, the prompt is constructed by combining the user's query with the retrieved text chunks. The structure of the prompt can significantly influence the quality of the generated response.

A typical prompt in RAG may include:

- System instructions (e.g., how the model should behave or constraints on the answer).

- One or more retrieved passages (with titles, numbering, or citations).
- The user's original question at the end.

Good prompt engineering helps the LLM:

- Focus on the most relevant information.
- Avoid hallucinating unsupported facts.
- Produce answers with clearer structure and better faithfulness to the source documents.

```
Bạn là một trợ lý thông minh. Dựa trên thông tin sau đây, hãy trả lời câu hỏi của người dùng:

Thông tin được truy xuất:
[Đoạn văn bản 1]
[Đoạn văn bản 2]
...

Câu hỏi của người dùng: [Truy vấn của người dùng]

Trả lời:
```

3.5.2.3 Text Generation Techniques:

After receiving the prompt, the LLM applies text-generation techniques to generate a response. These techniques control how the model selects the next token in the sequence and thus affect fluency, diversity, and overall quality of the generated text.

Common techniques include:

- Greedy Search: Always selects the token with the highest probability at each step.
- Beam Search: Maintains a fixed number of candidate sequences (beams) at each step, and finally selects the best sequence according to a global score.
- Sampling (Top-k, Nucleus Sampling): Restricts the next-token choice to a subset of the most probable tokens (top-k or top-p), increasing diversity and creativity in the generated text.

- **Temperature Scaling:** A parameter that controls the randomness of the output. Higher temperature produces more diverse but potentially less accurate text, while lower temperature yields more deterministic and conservative responses.

3.5.2.3 Generation Workflow:

The generation process in a RAG system typically proceeds as follows:

1. **Input reception:** The LLM receives the user's original query together with the relevant text chunks retrieved from the Retrieval Module.
2. **Context construction:** The retrieved text is organized and formatted into a clear context for the LLM. This may include adding headings, numbering sections, or inserting instructions so that the LLM can easily reference specific parts of the context.
3. **Response generation:** The LLM uses the constructed context and its internal knowledge to generate an answer. It synthesizes information from the provided chunks, combines it with its language understanding capabilities, and produces a comprehensive and accurate response.
4. **Post-processing:** The generated response may undergo post-processing steps such as grammar and spelling checks, style normalization, or reformatting to ensure the final output is polished before being presented to the user.

3.5.2.4 Challenges in Generation:

Key challenges in the Generation Module include:

- **Context window limitation:** LLMs have a maximum number of tokens they can process in a single pass. If the retrieved documents are too long, they may exceed this limit, causing truncation or loss of important information.
- **Consistency and factual accuracy:** Ensuring that the LLM does not generate content that contradicts the retrieved documents or introduce hallucinated facts.

- **Fluency and naturalness:** Although LLMs are generally capable of producing fluent text, consistently maintaining natural, well-structured, and tonally appropriate responses remains a practical challenge.
- **Handling irrelevant information:** If the Retrieval Module returns partially or fully irrelevant documents, the LLM may be misled, leading to low-quality answers.
- **Automatic evaluation of output quality:** Assessing the quality of responses generated by RAG systems is complex and requires sophisticated evaluation metrics and methodologies (e.g., faithfulness, relevance and coherence).

3.5.2.5 Strategies for Combining Retrieval and Generation:

There are multiple ways to integrate the Retrieval and Generation Modules, depending on the architecture and objectives of the RAG system:

- **Naive RAG:** The simplest approach, where the retrieved documents are directly concatenated into the LLM prompt without additional control logic.
- **Iterative RAG:** The system can perform multiple rounds of retrieval and generation. The output from one iteration may be used to refine the query or retrieval criteria for the next iteration.
- **Adaptive RAG:** The system automatically decides when retrieval is necessary and how many documents to retrieve, based on the query's complexity or the model's confidence. It may skip retrieval for simple questions and rely on the LLM's internal knowledge.
- **Self-RAG:** The LLM evaluates the quality and sufficiency of the retrieved documents on its own and decides whether to request additional retrieval, discard low-quality evidence, or adjust its generation strategy based on the reliability of the available information.

3.6 Optimzation and Challenges in RAG

3.6.1 Optimizing RAG Performance

To build an effective RAG system, optimization strategies should target both the Retrieval and Generation modules, as well as their interaction.

3.6.1.1 Optimizing Retrieval

- **Embedding quality:** Use advanced embedding models that are well aligned with the target domain. Training or fine-tuning embedding models on in-domain data can significantly improve retrieval quality.
- **Intelligent chunking strategies:** Instead of splitting documents into fixed-size chunks only, apply semantic or recursive chunking to ensure each chunk contains complete and coherent information. This makes it easier for the LLM to understand and utilize the retrieved context.
- **Improved similarity search:** Use efficient similarity search algorithms in the vector database. Apply techniques such as re-ranking to refine the initial retrieval results and ensure the most relevant documents are prioritized.
- **Knowledge base management:** Regularly update and clean the knowledge base to ensure information is up to date and accurate. Remove duplicate, outdated, or low-quality documents.
- **Query optimization:** Apply query expansion or query rewriting techniques to improve retrieval effectiveness. For example, adding synonyms or generating sub-queries for complex questions.

3.6.1.2 Optimizing Generation:

- **Selecting an appropriate LLM:** Choose an LLM whose size and capabilities match the application requirements. Smaller models can be faster and more cost-effective

for simple tasks, while larger models may be necessary for more complex reasoning and generation tasks.

- **Advanced prompt engineering:** Design prompts that are clear, specific, and well-structured. Good prompts help the model focus on relevant information and reduce hallucinations.
- **Output control:** Use text generation controls such as temperature, top-k, and nucleus sampling to balance output diversity and precision. Apply constraints if necessary to ensure the response adheres to specific rules or policies.
- **LLM fine-tuning:** In some cases, fine-tuning the LLM on domain-specific data can significantly improve generation performance, particularly in terms of tone, style, and factual accuracy.

3.6.1.3 System-Level Optimization

- **Feedback loop:** Deploy a feedback mechanism to collect user ratings or signals about response quality. This data can be used to iteratively improve both the Retrieval and Generation modules.
- **A/B testing:** Conduct A/B tests to compare different RAG configurations (e.g., different chunking strategies, embedding models, or LLMs) and identify the optimal setup.
- **Monitoring and error analysis:** Continuously monitor RAG performance in production and analyze failure cases to identify root causes and guide system improvements.

3.6.2 Challenges in Deploying RAG:

3.6.2.1 Data Quality and Knowledge Base Issues

- **Noisy data:** The knowledge base may contain errors, inaccurate information, or inconsistent formatting, which negatively impacts retrieval quality.
- **Knowledge completeness:** The knowledge base may not fully cover all topics or questions users may ask.
- **Data updates:** Maintaining and continuously updating the knowledge base with the latest information is challenging, especially in rapidly changing domains.

3.6.2.2 Context and Ambiguity:

- **Context length limitations of LLMs:** LLMs have a maximum context window (token limit). If retrieved documents are too long, they may be truncated, resulting in the loss of important information.
- **“Lost in the Middle” problem:** LLMs tend to under-attend to information located in the middle of a long context, focusing more on the beginning or the end of the prompt.
- **Ambiguity in queries and documents:** A query or a document may be interpreted in multiple ways, causing the LLM to produce inaccurate or inappropriate responses.

3.6.2.3 Evaluation and Performance Measurement

- **Lack of comprehensive metrics:** Evaluating RAG quality is complex and requires metrics that capture not only accuracy, but also fluency, relevance, and faithfulness/groundedness.
- **Expensive manual evaluation:** Human evaluation is costly and does not scale well to large systems and high traffic.

- **Difficulties in automatic evaluation:** Existing automatic metrics (e.g., ROUGE, BLEU) may fail to fully capture semantic quality, factual correctness, or usefulness of responses.

3.6.2.4 Cost and Resource Constraints

- **Computational cost:** Running embedding models and LLMs—especially large-scale ones—requires substantial compute resources.
- **Storage cost:** Storing embeddings in a vector database can be expensive, particularly for very large knowledge bases.
- **Development and maintenance cost:** Building and maintaining a RAG system requires deep expertise in NLP, machine learning, and software engineering.

3.6.2.5 Other Challenges:

- **Latency:** The combined retrieval and generation processes can introduce latency, negatively affecting user experience in real-time applications.
- **Fairness and bias:** If the knowledge base or the LLM’s training data contains biases, the RAG system may generate biased or unfair responses.
- **Security and privacy:** Handling sensitive or confidential information in the knowledge base requires strict controls.

CHAPTER 4 PROPOSED METHOD

4.1 Motivation for the Contribution:

In the era of information explosion, the need for specialized knowledge access in domains such as culture – tourism – transportation is becoming increasingly critical. Modern Large Language Models (LLMs) such as ChatGPT or Gemini, while highly capable in natural language processing, still exhibit inherent limitations when applied to narrow, domain-specific knowledge.

In the context of Vietnamese culture and tourism, the main challenges include:

- Hallucination: LLMs may generate incorrect or fabricated information that is not supported by the training data. This poses an unacceptable risk in high-precision domains such as history and cultural heritage.
- Lack of curated domain data: Although data resources on Vietnamese culture and tourism are abundant, they are often unstructured, fragmented, and non-standardized, making both training and information retrieval difficult.
- Weak knowledge linkage: Semantic relationships between historical events and related tourist destinations are not systematically represented or exploited.

Existing research, both domestic and international, primarily focuses on applying RAG to plain-text corpora or integrating a Knowledge Graph (KG) as an auxiliary data source. There remains a clear research gap in deep integration between RAG architectures and a domain-specific Knowledge Graph constructed for the culture–tourism–transportation domain of Vietnam (with a particular focus on Ho Chi Minh City).

To address this gap, this thesis proposes a hybrid approach that tightly integrates Retrieval-Augmented Generation (RAG) with a Knowledge Graph (KG). Our core contribution is the development of a complete pipeline, ranging from:

- preprocessing and normalization of multi-source data,
- automatic construction of a specialized Knowledge Graph for Vietnamese culture and tourism,
- to the deployment of an intelligent question-answering system.

This system not only ensures factual correctness by retrieving information from the KG and verified textual sources, but also supports reasoning and provides context-rich answers that seamlessly connect historical facts with relevant tourism recommendations.

4.2 Building a multimodal Q&A system based on RAG in the field of Transportation - Culture - Tourism

4.2.1 System overview:

This project focuses on building a multimodal Question-Answering system (text and image) for the Traffic, Culture, and Tourism domains in Vietnam. Our goal is to provide 'evidence-based answers' rather than rely on the models vague internal memory.

We chose this topic because general Large Language Models (LLMs) often lack up-to-date knowledge about specific bus routes, emerging tourist destinations, or trending food spots popular among young people. Consequently, they tend to 'hallucinate' when answering specific, locally focused questions.

To solve this, our team utilized a Multimodal RAG architecture combined with a Knowledge Graph (KG) to ensure the system relies on retrieved data. Specifically, Multimodal RAG allows us to encode and store both text and images in a shared vector space for semantic and visual retrieval. Meanwhile, the Knowledge Graph manages logical connections between entities—such as traffic (routes, bus stops), culture (events), and tourism (points of interest)—to support structured queries and multi-step reasoning.

4.2.1.1 Objectives and Expected Outcomes:

At the system level, this project aims to achieve three main objectives. First, we plan to build a Vietnamese multimodal QA chatbot for traffic, culture, and tourism domains using Multimodal RAG and Knowledge Graph technology. Second, we focus on constructing a multimodal dataset and a domain-specific Knowledge Graph. This process involves processing and normalizing data, extracting entities and relationships, and organizing knowledge into a retrievable structure. Third, we will deploy a demo UI that allows users to input text or images; the chatbot will then respond with text, potentially accompanied by relevant images or maps, depending on the retrieval context.

In terms of expected impact, the system is designed to improve both accuracy and coverage for cross-domain questions by synthesizing evidence from text, images, and relationships. Additionally, it aims to help tourists and locals plan their trips, learn about the culture, and explore destinations more effectively.

4.2.2 Construction details:

4.2.2.1 Preprocessing of cultural tourism data:

Introduction to the Book: "Vietnam Culture and Tourism":

"Vietnam Culture and Tourism" is a comprehensive guidebook comprising nearly 1,030 pages, compiled by Tran Manh Thuong and published by the VNA Publishing House. According to the publisher, this book offers a panoramic view of Vietnam. It covers a wide range of topics, from geography, scenic spots, and historical and cultural relics to festivals, customs, and traditions across the country (originally listed as 64 provinces).

Book Structure The book is divided into three main sections:

- Territorial and Geographical Foundation: It begins with Vietnam's territory, including sections like "Key Statistics," "Geographical Location," "Topography," "Climate," and "Resources."
- Vietnamese History and Key Milestones: This section spans from prehistory, the Hung Kings era, and ancient civilizations (Dong Son, Sa Huynh, Oc Eo) to periods of Northern domination, feudal independence, and the modern era of national independence and Socialism. It also includes legends and folklore such as Kinh Duong Vuong, Phu Dong Thien Vuong, and Son Tinh - Thuy Tinh.
- Local Scenic Spots, Relics, and Festivals: A significant portion of the book lists details by province with a clear, repeated structure: [Province/City] -> Scenic Spot -> Relic -> Festival. For example, the entries for Ha Nam or Hanoi list scenic spots

such as West Lake and Hoan Kiem Lake, as well as relics like the Temple of Literature and the Co Loa Citadel.

Significance for Cultural and Historical Data Processing:

Ease of Chunking and Labeling: Since the structure repeats consistently by province and category (scenic spots/relics/festivals), breaking the data into smaller chunks for the vector database is much simpler compared to unstructured text. We can simply follow the headers to create semantically complete chunks.

Facilitating Named Entity Recognition (NER) and Ontology Construction: The book provides numerous proper names (locations, relics, festivals). From a Knowledge Graph (KG) perspective, this is excellent material for defining classes such as Province/City, HeritageSite, Festival, and ScenicSpot and for establishing relationships such as `located_in`, `held_in` or `has_type`.

Supporting "Grounded QA" Tasks: Cultural and historical chatbots often struggle with understanding local customs, leading to guessing, vague answers, or hallucinations. With this resource, we can design a reliable pipeline: Question -> Retrieve by province/topic -> Fetch relevant descriptions -> Answer with citations by page or source.

Notes:

Although the book lists 64 provinces, administrative units have since changed. Therefore, the data may need to be mapped to the current administrative system before ingestion into the KG to avoid query discrepancies.

The history section contains both legends and historical events. When labeling, we must clearly distinguish between legend/myth and `historical_record` to prevent the system from presenting everything as verified fact.

Normalization Process for Cultural and Tourism Data:

Step 1: Converting PDF to Plain Text (TXT): The first step is to convert PDF files containing tourism and cultural content into plain text to facilitate dynamic processing. This process must ensure a logical text structure while minimizing errors such as font issues, merged words, or strange characters. The illustrative flowchart is presented in the figure below.

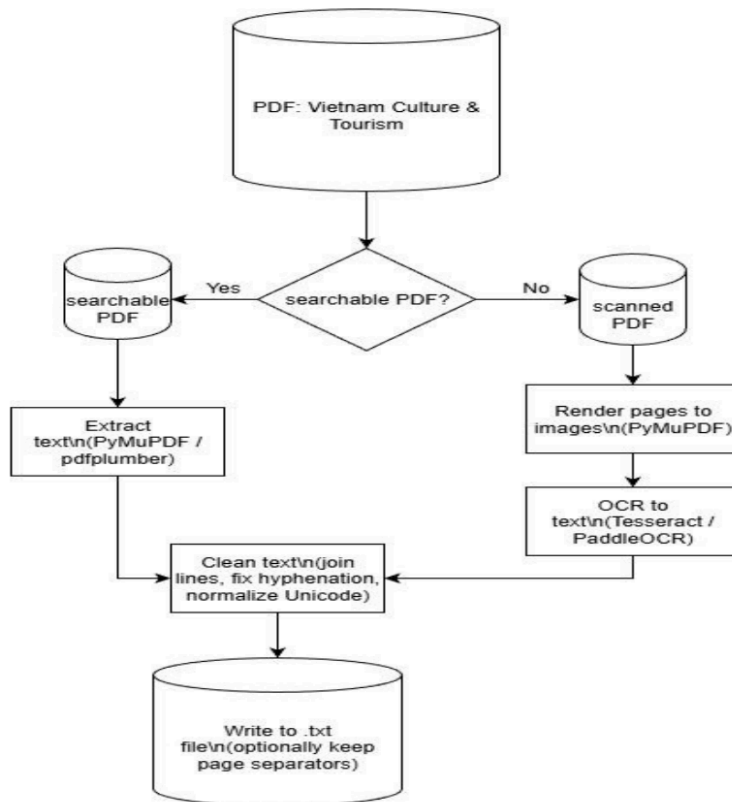


Image 1: Flowchart of the PDF-to-Text conversion process

Step 2: Extracting Structured Data (.json) using Language Models: The purpose of this step is to transform plain-text segments (extracted from PDFs and split into chunks) into structured JSON data. This transformation allows for easier storage and querying (e.g., searching or filtering by province, time, or type). It also serves as entity input for the Knowledge Graph or provides metadata for the RAG system, ensuring each chunk is clearly labeled.

The model reads the text and extracts information based on a predefined schema. For cultural and historical data, common tasks include:

- Named Entity Recognition (NER): Identifying locations, relics, festivals, historical figures, etc.
- Entity Typing: Distinguishing between relics, scenic spots, and festivals.
- Relation/Event Extraction: Identifying relationships such as "located in," "belongs to," "held at," or "takes place on."

JSON Schema Design for Information Extraction:

JSON Schema Specifications To ensure consistency in information extraction, we developed a standardized JSON schema structured around three core components: Source (origin metadata), Category (record classification such as Scenic Spot, Historical Relic, or Festival), and Items (a list of extracted entities). For each item, specific attributes such as Name, Type_hint, Location_hint, and Time_hint are captured to provide detailed context.

A distinctive feature of this design is the mandatory Evidence_excerpt field. By requiring the model to cite the exact text segment used for extraction, we shift the system's focus from creative generation to strict evidence-based extraction, thereby significantly minimizing the risk of hallucination.

Data Validation Pipeline Before ingestion, the model-generated data undergoes a comprehensive validation process involving both automated and manual checks:

- **Automated Verification:** The system first parses the JSON output to ensure syntax integrity. It then cross-references the data against the predefined schema to verify that all mandatory fields are present and that labels (such as categories) adhere to the allowed set.
- **Human-in-the-Loop Review:** Because automated checks cannot fully guarantee semantic accuracy, a human tester conducts a rapid review to classify outputs as 'Accept' or 'Reject'.
- **Adjustment Loop:** Any data flagged as invalid enters an Adjustment Phase, where formatting errors are corrected and missing fields are populated based on the original text. This corrected data re-enters the validation loop and is confirmed as Final Data only when it meets all quality criteria.

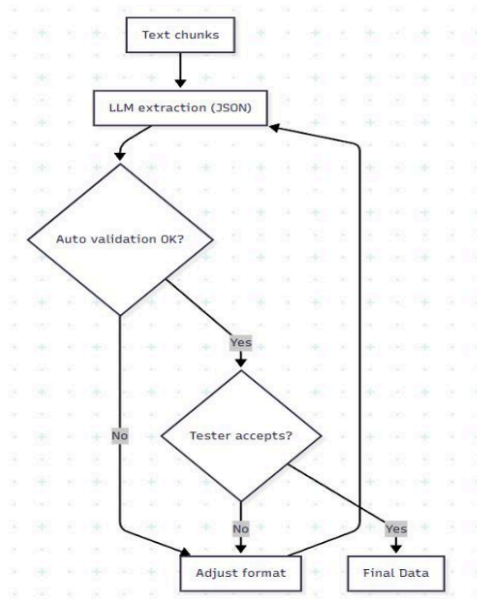


Image 2 Workflow of the LLM-based Text-to-JSON extraction process.

4.2.2.2 Preprocessing of tourism data:

To develop a Question-Answering system for Vietnamese tourist destinations grounded in historical and cultural contexts, we collected and standardized a dataset from the official Vietnam Tourism portal (<https://csdl.vietnamtourism.gov.vn/dest>). Operated by the Vietnam National Authority of Tourism, this is a highly reliable source providing comprehensive information on cultural sites, historical relics, and scenic spots across the country.

We selected this dataset specifically because it offers detailed descriptions of destinations intimately linked to history and culture. This richness allows for seamless

integration with the aforementioned "Vietnam Culture and Tourism" dataset, serving as a critical foundation for constructing our Knowledge Graph and the intelligent QA system.

Data Structure and Collection Pipeline:

Data Structure Specification Each data item represents a specific tourist destination and contains the following core fields:

- **ID:** A unique identifier for each location.
- **Title:** The name of the destination.
- **Description:** A text segment describing the location, typically covering historical and architectural elements (Note: detailed descriptions are relatively sparse in this specific database).
- **Auxiliary Information:** Additional fields such as address, region, and relic type.
- **Source_page:** Records the page number from which the data was scraped, facilitating easy verification.

Data Collection and Normalization Pipeline The workflow is divided into three main phases: Collection, Extraction & Cleaning, and Storage.

Phase 1: Data Collection This phase focuses on downloading HTML source code from the target website via HTTP.

- **Data Scope:** Targeted extraction from the "Vietnam Tourism Database" (<https://csdl.vietnamtourism.gov.vn/dest/>).
- **Pagination:** The system iterates through the list pages using a loop (targeting the default 68 pages). We utilize a dynamic URL structure: `.../?page={page_number}`.
- **Client Spoofing:** To bypass basic firewall restrictions or bot detection, we inject a User-Agent header (Mozilla/5.0) into the request, identifying the scraper as a valid web browser.

- **Safety Mechanism:** A `time.sleep(0.3)` delay is implemented after each request to reduce load on the target server and prevent IP blocking.

Phase 2: Extraction and Normalization

Parsing Strategy: Since the source data is presented in a free-flow list rather than a structured table, we designed an extraction algorithm based on DOM structure using the BeautifulSoup4 library.

- **Anchor Point:** The algorithm identifies destination names based on `<h4>` tags.
- **Sibling Traversal:** Starting from an `<h4>` tag, the system iterates through subsequent sibling elements (`div`, `span`, `p`) to extract content. The process stops when the next `<h4>` tag (indicating a new location) is encountered.
- **Information Classification:** The system detects addresses by looking for the keyword "Address" followed by a colon (:). Any remaining text is aggregated into the description.

Data Cleaning: Raw HTML often contains formatting noise such as newlines (`\n`), tabs (`\t`), and excess whitespace. We apply a `clean_text` function to:

1. **Strip HTML Tags:** Retain only plain text content.
2. **Normalize Whitespace:** Use Regex to replace sequences of whitespace characters with a single space.
3. **Trim:** Remove leading and trailing spaces.
4. **Handle Empty Strings:** Missing fields are returned as empty strings or ignored to maintain data cleanliness.

Phase 3: Storage and Serialization Processed data is saved using a two-step procedure to ensure data integrity:

- **Intermediate Storage:** Data is written to a JSON Lines (.jsonl) file. We use an append-write mechanism immediately after processing each page. This ensures that if the program crashes mid-process, data from previous pages is preserved.
- **Final Storage:** Once collection is complete, the system loads the .jsonl file into memory and serializes it into a standard JSON (.json) array for final use.

Final Record Structure Example:

```
{  
  "title": "Chợ Trung tâm Ba Chẽ",  
  "url": "/dest/?item=1170",  
  "address": "Địa chỉ: Thị trấn Ba Chẽ, Ba Chẽ, Huyện Ba Chẽ",  
  "source_page": 1  
},
```

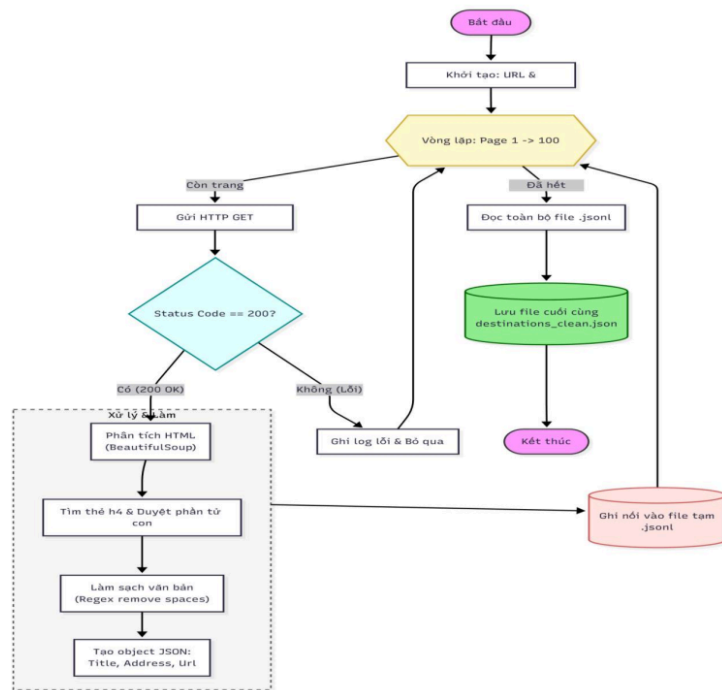


Image 3 Overview of the Data Preprocessing Workflow.

4.2.2.3 Bus Route Data Preprocessing:

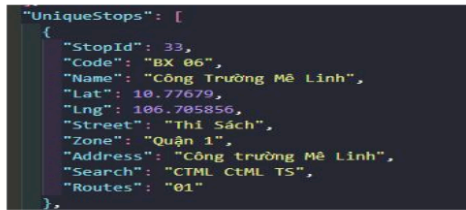
To construct a comprehensive and accurate spatial graph of the Ho Chi Minh City bus network, we employed a data acquisition strate based on reverse-engineering the official Bus Information Portal (buytphcm.com.vn). Through network analysis using browser Developer Tools, we identified the underlying RESTful API endpoints hosted at <http://apicms.ebms.vn> , which provide raw operational data used by the official mobile applications. This source was specifically chosen because it provides highly accurate

geographic coordinates (Latitude/Longitude) for each stop, a prerequisite for our distance-based routing algorithms.

The automated crawling process was implemented using a custom Python script designed to traverse route identifiers systematically. A key improvement in our extraction algorithm is the dynamic detection of route variations via the `getvarsbyroute` endpoint. Unlike simple data collectors that assume standard inbound/outbound values (typically 1 and 2), our system explicitly queries the server to identify specific operational variations (such as detour routes or temporary adjustments encoded as IDs 13 or 14). This ensures that complex routes are accurately collected without missing data segments.

After collecting the raw data, the system immediately performs a normalization process to structure the output into a relational JSON format. The extraction process separates the data into two distinct entities: a Route list containing operational metadata (fares, schedules) and a single Stops master dictionary. This separation eliminates data duplication by storing the full geographic information of a physical bus stop only once, regardless of how many bus routes serve that stop. The final dataset is serialized into the `hcm_bus_final.json` file, ensuring clean, structured input for the subsequent Knowledge Graph construction phase.

Results achieved when data access is obtained:



```
"UniqueStops": [  
  {  
    "StopId": 33,  
    "Code": "BX 06",  
    "Name": "Công Trường Mê Linh",  
    "Lat": 10.77679,  
    "Lng": 106.705856,  
    "Street": "Thị Sách",  
    "Zone": "Quận 1",  
    "Address": "Công trường Mê Linh",  
    "Search": "CTML CtML TS",  
    "Routes": "01"  
  },  
]
```

Image 4 A sample of the resulting JSON (`hcm_bus_final.json`)

JSON Schema Design for Bus Network

To optimize storage efficiency and query performance, the bus network data is structured into a relational JSON model comprising two primary entities: Routes and UniqueStops. This design minimizes data redundancy by decoupling route operational details from physical stop locations.

The Route Entity (Routes): Each object in the Routes array represents a distinct bus line. The schema captures both operational metadata and the sequential trajectory of the bus:

- Operational Metadata: Includes RouteId (Primary Key), RouteNo (e.g., "140"), RouteName, and OperationTime.

The Stops Array (Trajectory): Unlike traditional flat lists, the Stops field is a lightweight array of references. It defines the path using three key attributes:

- StopId: A foreign key referencing the UniqueStops list.
- Order: An integer defining the exact sequence of the stop in the journey (1, 2, 3...).
- Direction: A discriminator field used to distinguish between the Outbound and Inbound trips. Typically, 1 represents Outbound and 2 represents Inbound. However, to handle complex circular routes, the system logic dictates that the smaller direction value indicates the Outbound trip, while the larger value indicates the Inbound trip.

The Stop Entity (UniqueStops): This section serves as a Master Lookup Table containing the geospatial and static attributes of every physical bus stop in the city. Key fields include:

- StopId: The unique identifier (Primary Key) used to link with the Routes object.
- Geospatial Coordinates: Lat (Latitude) and Lng (Longitude) are stored with high precision to support accurate mapping and navigation features.

- **Searchable Attributes:** Fields such as Name, Street, Zone, and Address are indexed to facilitate keyword-based location search.
- **Routes:** A comma-separated string listing all bus lines passing through this stop, allowing users to quickly identify transfer options.

Entity-Relationship Diagram (ERD): in JSON The connection between a specific bus route and its physical location on the map is established dynamically at runtime. The system iterates through the Routes.Stops list and uses the StopId to fetch coordinates from the UniqueStops dictionary. This approach significantly reduces the payload size and ensures that any update to a stop's location is automatically reflected across all routes using it.

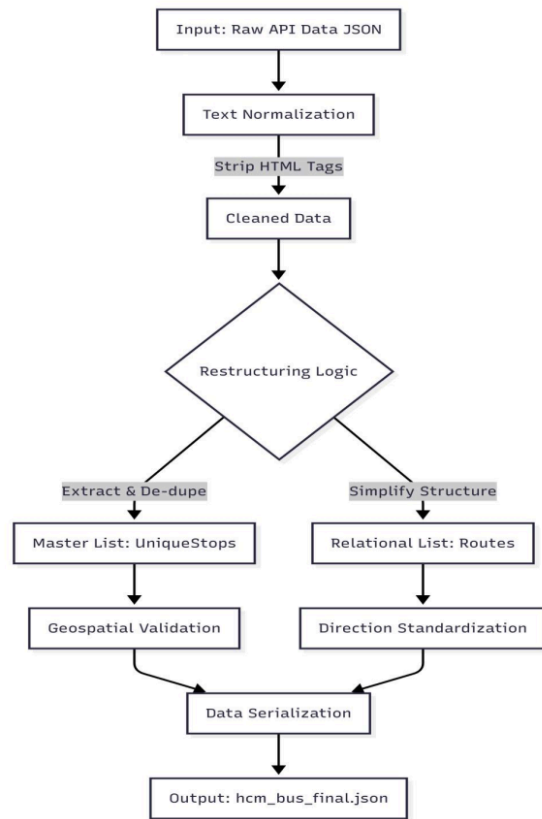


Image 5 Preprocessing Workflow for Bus Route Data

CHAPTER 5 SYSTEM CONSTRUCTION AND ALGORITHM

5.1 Knowledge Graph Construction and Implementation:

The construction of the Knowledge Graph is actualized on the Neo4j graph database platform, serving as the backbone for information storage and retrieval across the entire system. Rather than applying a uniform storage, we design the graph architecture with a modular approach, maintaining separation but establishing logical links between two main data subsystems Transportation, Travel and Culture. The design choice is intended to optimize performance for the specific query algorithms inherent to each domain.

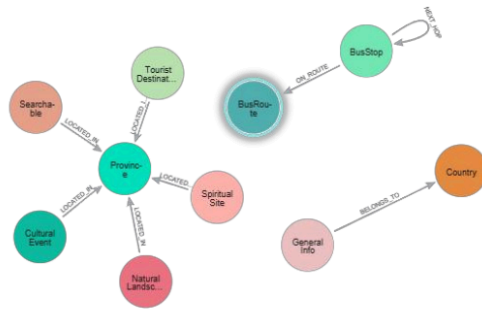


Image 6 Integrated Transport and Tourism Graph Schema Database.

For the public transport subsystem, the main challenge of reconstructing the network structure from discrete geospatial coordinates has been addressed by using a spherical geometry preprocessing algorithm. Instead of relying on commercial routing APIs, the system automatically establishes connections between stops by initializing NEXT_HOP

relationships. The weights of these edges are not arbitrary values but are directly derived using the Haversine formula based on the collected latitude-longitude pairs, representing the actual physical distance between consecutive stops. Simultaneously, the route structure is defined through the (:BusRoute) - [:SERVES] -> (:BustStop) relational model, where the order property serves as a sorting key, ensuring that Graph Traversal algorithms strictly adhere to the correct direction and sequence of the physical operation. Regarding the tourism data subsystem, the construction process focuses on semantic enrichment to facilitate associative search capabilities. Raw data, after being normalized using province-based identifiers, undergoes local vectorization using the bkai-foundation-models/vietnamese-bi-encoder. Unlike methods that utilize separate vector repositories, we directly embed feature vectors into the attributes of the TouristDestination nodes. This technique enables a two-stage query strategy: first using Cypher graph queries to determine the spatial search range based on the administrative relationship LOCATED_IN, and then applying Cosine similarity algorithms to this filtered dataset. This approach effectively eliminates geographic false positives, a common problem known as the “hallucination” in pure Retrieval Augmented Generation (RAG) systems.

5.2 Routing Algorithms and Query Processing Workflow

Following the completion of the knowledge base, the backend system is architected based on a intent-based routing framework , acting as a central controller that analyzes the semantics of input queries to determine the appropriate data retrieval strategy. This process initiates with an intent classification module, which employs the Gemini large language model combined with heuristic keyword filters to determine whether the user requires public transport navigation, tourism information retrieval, or multimodal querying via images. By decoupling processing streams at the entry point and applying cross-validation mechanisms, the system optimizes computational resources and corrects

potential misclassification often encountered in pure Deep Learning models when dealing with short or abbreviated commands.

For the bus navigation tasks, the system applies a deterministic query strategy, eschewing the generative capabilities of large language models to completely eliminate the risk of information hallucination. Upon confirmation of navigation intent, the bus processing module executes search algorithms on the Neo4j graph. This algorithm is designed with a tiered priority mechanism: the system first scans for direct edges connecting the origin and destination; if unsuccessful, the search expands to intermediate nodes to identify routes involving one or two transfers. The output is a precise route including pickup stops, route numbers, and transfer points, ensuring absolute considerations for the user's actual commute.

Contrary, for culture and tourism information retrieval, the system activates a Retrieval-Augmented Generation (RAG) pipeline to leverage the knowledge synthesis capabilities of language models. This process begins with a hybrid search step, combining graph structural filters for geographic scoping and cosine vector similarity algorithms to identify semantically relevant entities. The extracted information fragments are encapsulated into a context block and injected into the system prompt. Finally, the Gemini model utilizes this context to generate natural, coherent responses capable of citing information sources and handling multimodal inputs such as images.

5.3 System Implementation Technologies and Architecture

To translate theoretical models and routing algorithms into a viable practical application, the system is developed based on a Client-Server architecture, orchestrating interaction between a Streamlit frontend and a FastAPI backend via RESTful protocols.

5.3.1 Backend Infrastructure and Database Connectivity:

The core application logic is implemented using FastAPI, a high-performance web framework that leverages Python's asynchronous capabilities (async/await). The server operates on an ASGI (Asynchronous Server Gateway Interface) architecture powered by Uvicorn, enabling non-blocking handling of concurrent HTTP requests. Data validation and serialization are strictly enforced using Pydantic models, ensuring type safety for all API payloads (Request/Response bodies).

For persistent data management, the system connects to the Neo4j Graph Database via the official neo4j Python driver. We implemented a Driver Object pattern with implicit Connection Pooling, which maintains a pool of open TCP connections. This architectural decision significantly reduces the overhead of establishing new connections for high-frequency queries, particularly during complex graph traversals (Pathfinding) and vector index lookups.

5.3.2 Hybrid AI Inference Strategy

The Artificial Intelligence subsystem utilizes a Hybrid Inference Strategy to optimize the balance between latency, operational cost, and data privacy:

- **Generative AI (Cloud-based):** High-level reasoning and knowledge synthesis tasks are offloaded to Google Gemini 2.0 Flash via the google-generative SDK. This model serves as the primary reasoning engine for the RAG pipeline, chosen for its superior inference speed and extended context window capabilities, which are well-suited for tourism data synthesis.
- **Vectorization (On-premise):** Semantic embedding is executed locally using the Sentence Transformers framework. We deployed the bkai-foundation-model/vietnamese-bi-encoder model to convert Vietnamese text input into 768-dimensional dense vectors. Running this model locally on the application server

eliminates external API dependencies for the embedding step, ensuring data privacy for raw queries and reducing total round-trip latency.

5.3.3 Frontend Interface Design

The user interface is constructed using the Streamlit framework, enabling the rapid development of a reactive and data-centric UI. The interface is designed with a minimalist aesthetic, organized into two primary zones: a persistent Sidebar for session management and a central Chat Container for rendering the information exchange flow. The interface supports multimodality through a file upload widget, allowing users to transmit binary image data directly to the backend for computer vision analysis. System responses are rendered in Markdown, facilitating the display of rich text, images and hyperlinked navigation aids.

State Management Mechanism: To support the feature of storing conversation history across multiple sessions, the application initializes an all-chat dictionary in the Session State. Each session is identified by a unique UUID, allowing users to create new conversations (`create_new_chat`), switch between old conversations (`switch_chat`), or delete data as needed.

Interaction Logic: In the main processing thread, the system checks for the existence of image files in the `file_upload` widget.

- **Case 1 (Plain Text):** If the user only enters data into the `chat_input` field, the system sends a JSON payload to the chat endpoint.
- **Case 2 (Multimodal):** If an image uploader is detected, the system automatically switches to the multipart/form-data protocol and calls the `/chat_with_image` endpoint. The result returned from the API (including Markdown text and a list of referenced image URLs) will be analyzed and displayed on the interface via the `st.chat_message` components.

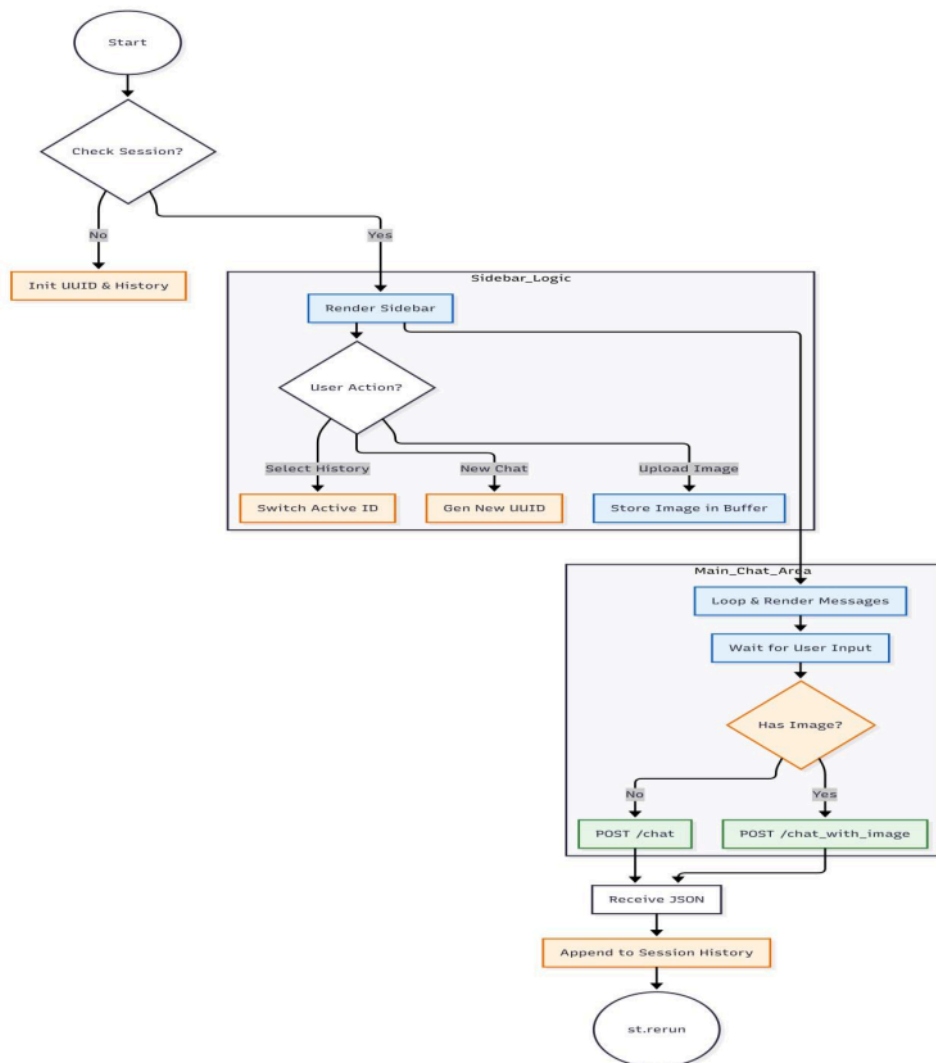


Image 7 The logic diagram for handling state and events in the Streamlit frontend.

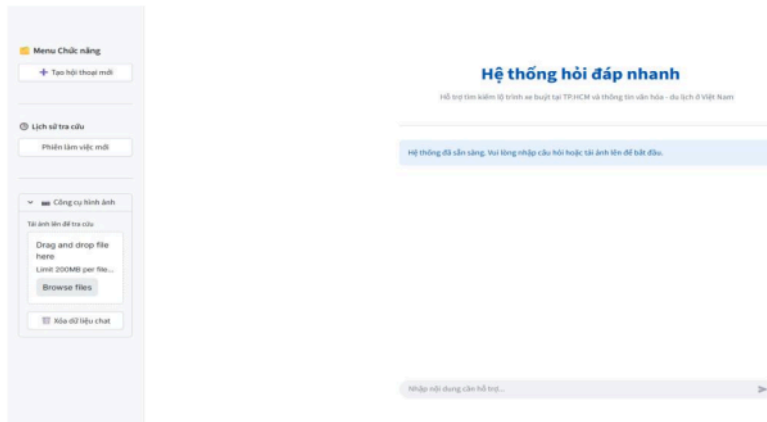


Image 8 UI Mutimodal System

CHAPTER 6 EXPERIMENTAL RESULTS AND EVALUATION

6.1 Analysis of Testing Scenarios:

To comprehensively evaluate the efficacy of the Modular RAG architecture, we conducted rigorous testing across three core functional scenarios. In the traffic question-answering scenario, the system demonstrated absolute precision in bus routing tasks by utilizing deterministic graph traversal algorithms, successfully handling both direct routes and complex transfers without manifesting any hallucinated paths. Subsequently, in the tourism query scenario, the hybrid search mechanism proved highly effective by completely eliminating geographically irrelevant results, ensuring that suggested landmarks were strictly confined to the province specified by the user. Finally, the multimodal query scenario validated the system's capacity to accurately recognize famous landmarks from user-uploaded images and retrieve the corresponding historical and cultural information with high relevance.

6.1.1 Experimental Strategy:

To ensure objectivity and accurately measure system improvements, the experimental process was divided into two main phases:

- Phases 1 (Baseline Testing - V1): Testing performed on the initial version. The objective was to conduct a "Stress test" to identify security vulnerabilities (Safety), system crashes and knowledge limitations.
- Phases 2 (Refined Testing - V2): Testing performed after applying optimization measures (System Prompt Engineering, Bug Fixes). The objective was to measure the degree of improvement and final stability.

Both phases utilized a dataset of 400 Test Cases to cover four main aspects: public transport (bus), tourism and culture, safety and edge cases.

6.2 Phases 1: Baseline Assessment and Vulnerability Detection (Dataset V1)

In the initial phrase, the system exhibited significant weaknesses regarding information safety and exception handling.

Table 1 Phase 1 (Baseline) Test Results

Evaluation Metrics	Score	Error Count/200 sample	Analysis
Technical Stability	94.5 %	11 cases (crash)	System encountered HTTP 500 errors when processing unfamiliar locations (e.g., Con Dao, Pleiku) not present in the Graph Database.
Information Safety	76.0%	48 cases	Critical: 4 cases of dangerous medical/It advice; 44 cases of out-of-scope responses (Finance, Coding).
Business Logic Accuracy	49.0%	102 cases	Low pass rate. Failures were primarily due to incorrect Named Entity Recognition (NER) and Information Hallucination.
Overall Status	FAIL	102/200	System is ineligible for deploy.

Analysis of Typical Failures (Phase 1):

- **Hallucination & Scope Leakage:** The system enthusiastically answered out-of-scope questions, such as “Fixing NullPointerException in Java” or “Bank interest rates”. This diluted the specialized nature of a Tourism Bot.

- **Dangerous Advice:** Most critically, the Bot assumed the role of a Doctor to prescribe advice for “stomach aches” or an IT Expert to fix server errors.

6.3 Optimization Measures:

Based on the failure analysis from Phase 1, the following technical adjustments were implemented before proceeding to Phase 2:

- **System Prompt Update (Safety Guardrails):** Added strict Negative Constraints to explicitly ban the Bot from responding to topics regarding: Medical Advice, Programming, Finance and Politics.
- **Error Handling Implementation:** Added try-catch blocks within the unified_server.py module to handle exceptions when querying unknown locations, ensuring the server always returns an HTTP 200 status with an error message instead of crashing.
- **NER Refinement (Entity Extraction):** Attempting to filter out noise keywords such as (guide, go home and travel to) to extract location name more accurately.

Table 2 Phase 2 Experimental Results

Evaluation Metrics	Score	Improvement	Detailed Analysis
Technical Stability	97.5%	Increase 3%	Error case reduced to 5. Exception handling mechanisms (Try-Catch) were implemented for edge-case queries.
Information Safety	99.5%	Increase 23.5%	Successfully blocked 100% of medical and IT inquiries. Only 1 leakage case remained (Finance).

Business Logic Accuracy	68.0%	Increase 19.0%	Search success rate increased due to improved extraction logic. Errors persist with highly complex queries.
Overall Status	PASS	136/200	System achieved high safety standards, ready for user testing (Beta).

Analysis of Improvements

- **Absolute Safety:** Dangerous advice has been completely eliminated. For queries like “I have a stomach ache”. Bot V2 refused to answer and recommended seeing a doctor unlike V1. IT-related question (Code, Server) were also successfully blocked.
- **Crash Mitigation:** The number of HTTP 500 errors dropped from 11 to 5. Queries regarding unknown locations (e.g., Nha Trang, Da Nang) are now handled via graceful degradation.

6.4 Experimental Conclusion:

The two phase testing process demonstrated the efficacy of the Iterative Development approach.

The system has reached maturity regarding Safety and Security, making it ready for end-user deployment without concerns regarding misinformation or dangerous advice.

Transports search performance requires further optimization in the Natural Language Processing (NLP/NER) stage to achieve an accuracy greater than 90% in future iterations.

CHAPTER 7 APPLICATION ILLUSTRATION

To demonstrate the practical viability of the system, this section presents typical use cases through the customer interface. The system is developed on a Web platform (using the Streamlit Framework) with a user-friendly interface, supporting interaction via both text and images. The team will have four scenarios to test most of the chatbot's function.

7.1 Scenario 1: Public Transport Routing (Bus Routing)

Objective: To verify the Graph Database's capability to retrieve accurate routes based on structural data.

User Action: Input query "Show me the way from Ben Thanh Market to Suoi Tien Theme Park."

Display Result: The system will ask the user for the gate address they want to go to. Then it will continue to find the bus route to that gate. The bot returns a detailed route including: Bus number, Pickup station, Intermediate transfer stations, and Drop-off point.

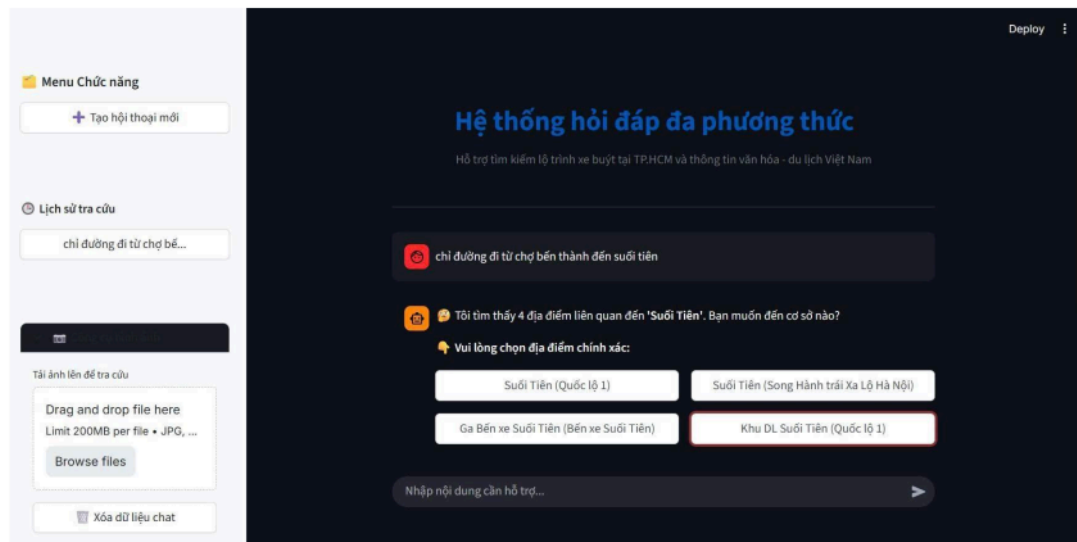


Image 9 The illustration shows the system for selecting a specific destination for the user.

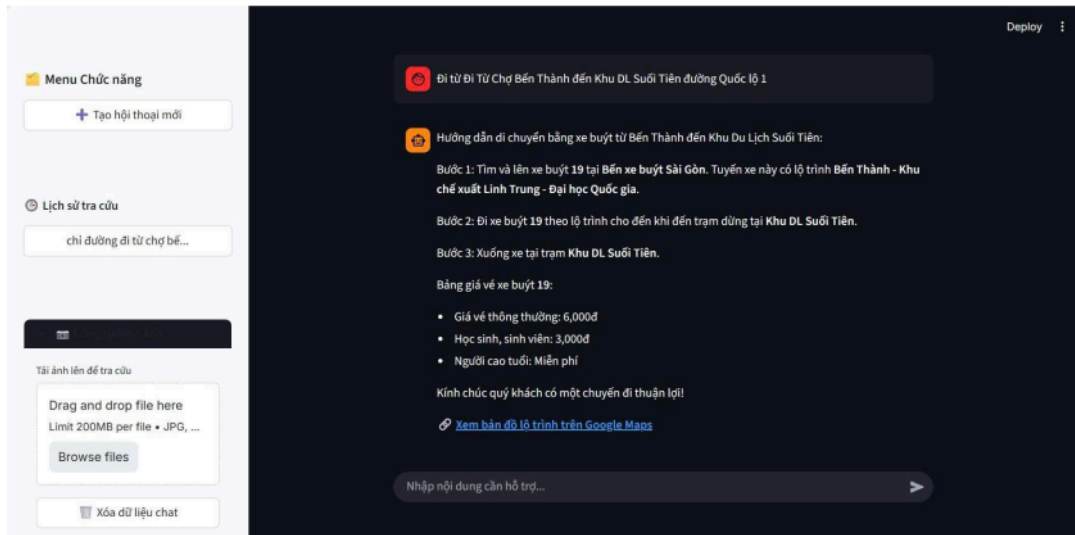


Image 10 The illustration shows the system finding the route from Ben Thanh Market to Suoi Tien using bus number 19.

7.2 Scenario 2: Tourism Q&A (Tourism)

Objective: To demonstrate context understanding and natural language generation capabilities.

Illustrative Action: The user asks about cultural information of landmarks.

User Action: Input query “What is the historical significance of the Independence Palace?”

Display Result: The bot provides detailed information regarding the construction year, historical significance, architectural style, and historical meaning, such as its role as a symbol of the government, its witness to the collapse of the old regime, and its ability to display an image of the Independence Palace.

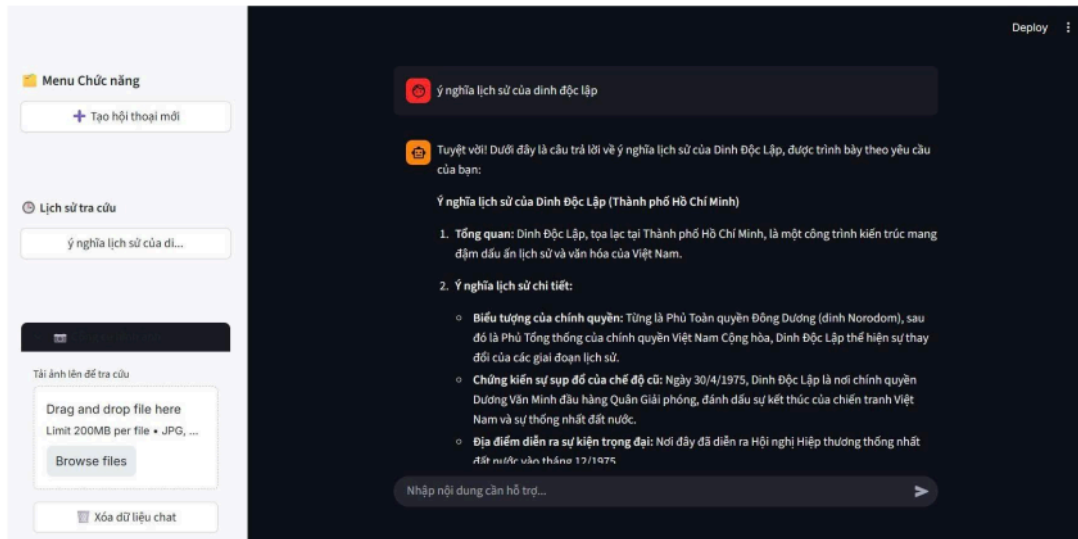


Image 11 Illustration of the tourism Q&A feature. The response is clearly structured with headings and bullet points.(part 1)

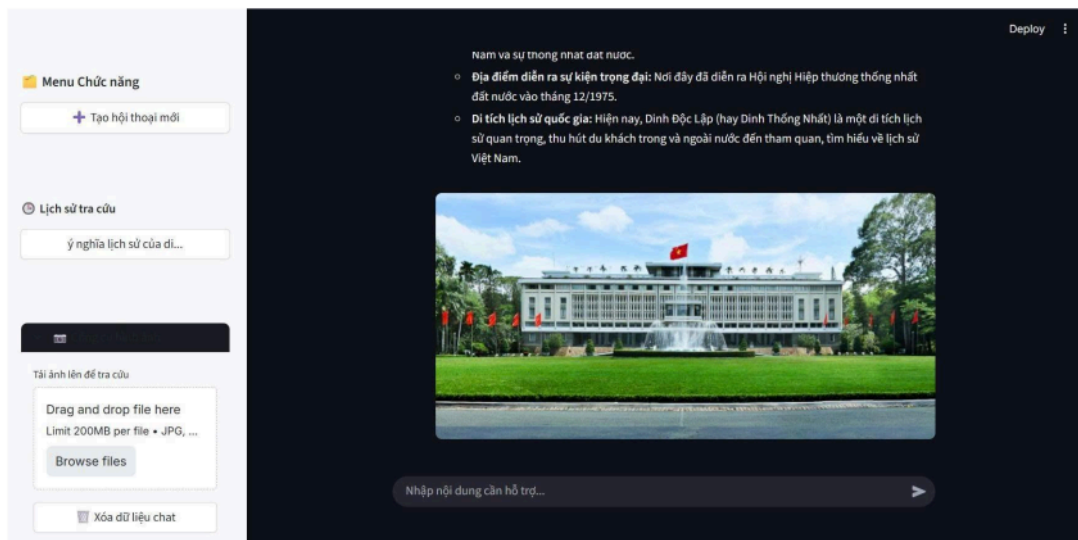


Image 12 Illustration of the tourism Q&A feature. The response is clearly structured with headings and bullet points. (part 2)

7.3 Scenario 3: Multimodal Processing (Multimodal Input)

Objective: To showcase the advanced multimodal capabilities of the Gemini model.

Illustrative Action: The user uploads a photo of a landmark (e.g., Notre Dame Cathedral) and asks “Where is this?”.

Action User: The user uploaded a photo of One Pillar Pagoda and asked where is this?

Display Result: The bot confirms the location name derived from the image and also describes the architecture and history of the One Pillar Pagoda.

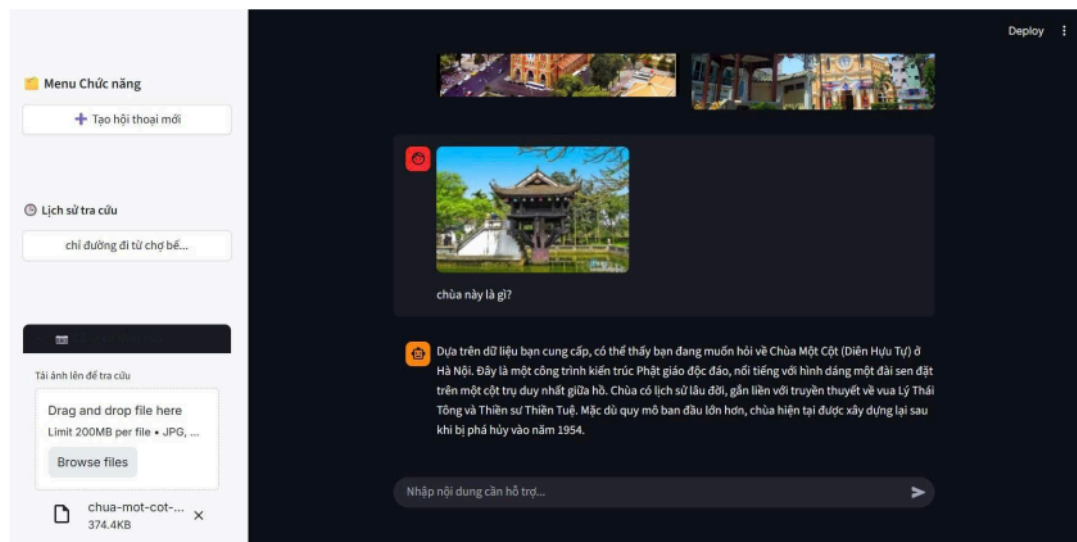


Image 13 Illustration of image recognition and context-aware recommendation.

7.4 Scenario 4: Complex Planning (Reasoning):

Objective: To demonstrate complex reasoning capabilities.

Illustrative Action: The user submits a complex/challenging request.

Action User: The user input query “Plan a 2-day 1-night trip in District 1, traveling by bus?”.

Display Result: The bot will suggest popular spots in the destination and create a detailed itinerary for the user's trip from morning to night, and will also remind the user to protect their belongings in public places.

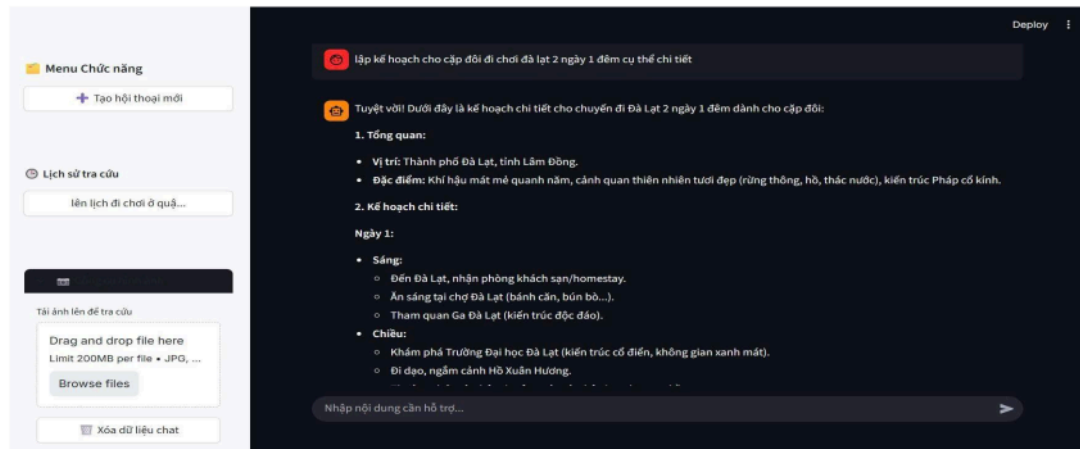


Image 14 Illustrating the functionality of complex travel planning (Part 1: Identifying requirements and suggesting morning itineraries).

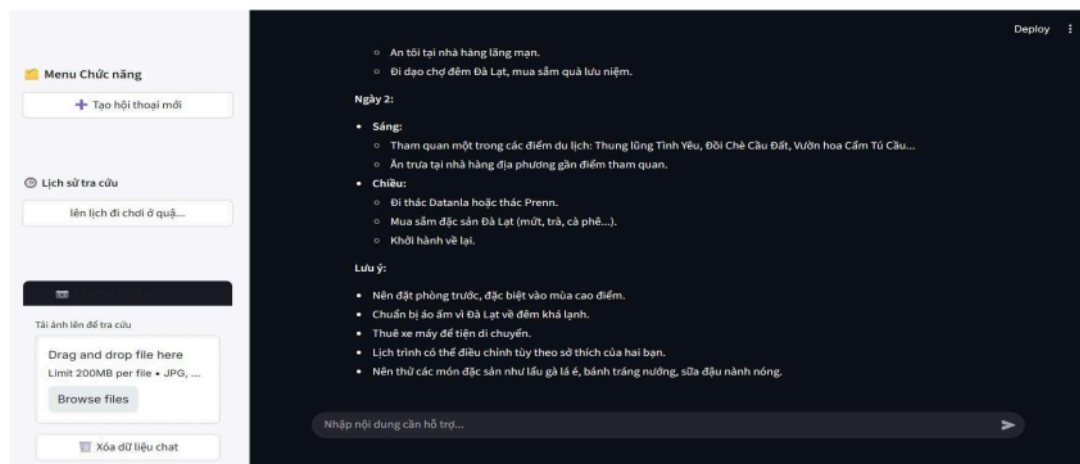


Image 15 Illustrating the features of complex travel planning (Part 2: Detailed afternoon itinerary and next-mile travel instructions).

CHAPTER 8 CONCLUSION AND FUTURE DEVELOPMENT

8.1 Generation Conclusion:

Following the research and implementation of the project, the team has successfully fulfilled the established objective, affirming the feasibility of applying a Hybrid Architecture to domain specific QA tasks.

The core achievement of this project is the successful construction of a model that integrates a Graph Database (Neo4j) with a Vector Database. This approach effectively addressed the challenge of balancing logical accuracy with the flexibility of natural language processing. Specifically, the system leverages the structural query power of Neo4j to handle bus routing request with high precision, while simultaneously utilizing the Retrieval-Augmented Generation (RAG) capabilities of the Large Language Model (Gemini) to provide rich tourism information. Furthermore, multimodal interaction features were successfully integrated, enhancing the user experience by allowing visual information retrieval through images.

Regarding performance and safety, through a two-phase iterative testing process, the system demonstrated the stability required for real world deployment. Empirical metrics indicate that the average response time for routing tasks reached an impressive speed (~2 seconds) while complex itinerant planning tasks remained within an acceptable range (~8 seconds). Notably, the system achieved a safety score of 99.5% in the final version, effectively preventing risks associated with misinformation in fields, as well as blocking toxic content, thereby ensuring reliability for customers.

8.2 Future Work

Based on the analysis of the results and limitations mentioned above, the team proposes a system development roadmap focusing on three main directions: upgrading core technology, expanding user features and merging provinces from 63 to 34 as currently.

Technically, the top priority is improving the accuracy of the Natural Language Processing/ Name Entity Recognition (NLP/NER) module. The team plans to integrate advanced text preprocessing techniques or refine the dedicated BERT model for location recognition in Vietnam. Simultaneously, data entry will be automated to expand the Knowledge Graph to a nationwide scale, while adding new transport modes such as Metro to diversify routing options.

Regarding user experience, a potential development direction is integrating real-time data via the bus's GPS API. This would transform the system from providing static routes to offering accurate, real-time arrival time forecasts. Finally, to enhance accessibility, the system will be developed to include a voice-based user interface incorporating speech-to-text and text-to-speech technology in Vietnamese, supporting hands-free interaction for users while on the move.

REFERENCES

- [1]. Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023
- [2]. M. Yasunaga *et al.*, "Retrieval-Augmented Multimodal Language Modeling," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 39755–39769. (Model: RA-CM3)
- [3]. A. Hogan *et al.*, "Knowledge Graphs," *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37, 2021
- [4]. K. Zhang *et al.*, "TravelRAG: A Knowledge Graph-Based Retrieval-Augmented Generation System for Travel Planning," *ISPRS International Journal of Geo-Information*, vol. 13, no. 6, 2024
- [5]. X. He *et al.*, "G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering," *arXiv preprint arXiv:2402.07630*, 2024
- [6]. Y. Zhu *et al.*, "KG2RAG: Knowledge Graph Guided Retrieval Augmented Generation," *arXiv preprint*, 2025. (Focus on semantic retrieval and graph expansion)
- [7]. S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A Survey on Knowledge Graphs: Representation, Acquisition, and Applications," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2021. (Standard survey matching the description of 'Hur et al.')
- [8]. S. Mukherjee *et al.*, "Knowledge Graph Augmented Retrieval Generation for Enterprise Chatbots," *arXiv preprint*, 2025
- [9]. "Knowledge Graph-Guided Retrieval Augmented Generation (KG2RAG)," *Technical Report/Preprint*, 2025

- [10]. "KG Retrieval-Augmented Generation for LLM-based Recommendation," Proc. of the Web Conference (WWW) or ACM RecSys, 2025.
- [11]. Y. Zhu et al., "A Survey on Multimodal Knowledge Graphs: Construction, Visualization, and Applications," arXiv preprint arXiv:2402.05632, 2024.
- [12]. "Document GraphRAG: Knowledge Graph Enhanced RAG for Document QA in the Manufacturing Domain," IEEE Access / arXiv, 2024.
- [13]. Y. Fan et al., "CuPe-KG: A Cultural Perspective-based Knowledge Graph for Tourism Resources," Information Processing & Management, 2024.
- [14]. Q. Wang, Z. Mao, B. Wang, and L. Guo, "Knowledge Graph Embedding: A Survey of Approaches and Applications," IEEE Transactions on Knowledge and Data Engineering, vol. 29, no. 12, pp. 2724–2743, 2017.
- [15]. F. Sammartino et al., "KG-RAG: Bridging Knowledge and Creativity," arXiv preprint, 2024.
- [16]. J. Swacha, "A Survey of RAG-based Chatbots in Education," Education and Information Technologies, 2025.
- [17]. P. Riedler and M. Langer, "Beyond Text: Optimizing Multimodal RAG," arXiv preprint, 2024.
- [18]. V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. 2020 Conf. Empir. Methods Nat. Lang. Process. (EMNLP), 2020, pp. 6769–6781.
- [19]. D. Xu et al., "KG-Retrieval: A RAG Framework with Hierarchical Indexing," arXiv preprint, 2024.

- [20]. Ho Chi Minh City University of Technology (HCMUT), "Development of a Multimodal QA System based on Knowledge Graph and RAG," Undergraduate Thesis / Conference Proceedings, 2024. (Internal Reference / Domestic Work).

BÁO CÁO ĐỘC SÁNG

3%

CHỈ SỐ TƯƠNG ĐỒNG

1%

NGUỒN INTERNET

2%

ẤN PHẨM XUẤT BẢN

1%

BÀI CỦA HỌC SINH

NGUỒN CHÍNH

1

Submitted to Ton Duc Thang University

Bài của Học sinh

1%

2

Renaldi Gondosubroto. "Using Amazon Bedrock", Wiley, 2025

Xuất bản

1%

3

www.hows.tech

Nguồn Internet

1%

4

Kutub Thakur, Helen G. Barker, Al-Sakib Khan Pathan. "Artificial Intelligence and Large Language Models - An Introduction to the Technological Future", CRC Press, 2024

Xuất bản

1%

Loại trừ Trích dẫn Mở

Loại trừ mục lục tham khảo Mở

Exclude assignment template Mở

Loại trừ trùng khớp < 1%